

Microservices

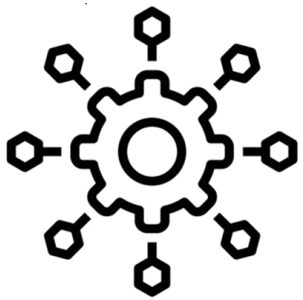
Microservicios

Universidad de La Sabana

FACULTAD DE INGENIERÍA

Universidad de La Sabana
FACULTAD DE INGENIERIA

Agenda



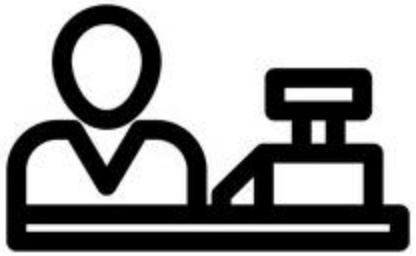
Microservices I

- Definitions
  - What is a service?
  - Motivation: Big Ball of Mud, Scale Cube
  - Comparison
    - Motivated from Architecture Attributes ?
    - by Martin Fowler
    - by Azure
  - Evolution from SOA
- Why not
  - Monolith First
  - YAGNI
  - Cognitive Bias: Prime Video Case
- Toolset
  - 12 Factors
- Methodology I: DDD
  - DDD and Microservices
  - Pattern Language: Mindmap
  - Definitions
    - Domain
    - Model
    - Ubiquitous Language
    - Bounded Context
  - Azure Proposal
    - P1: Analysize Domains
    - Eric Evan's Interview
    - P2: Define bounded Context
    - P3: Tactical DDD
    - P4: Identity microservices
  - Toolset
    - Conway's Law
    - Modeling
      - Event Storming
      - Event Modeling
    - Example of Modeling
- Study Cases
  - Latam Logistic Company
  - National Bank I
  - National Bank II

Universidad de La Sabana
FACULTAD DE INGENIERIA

What is a service ?





C-S Model

LB





V1



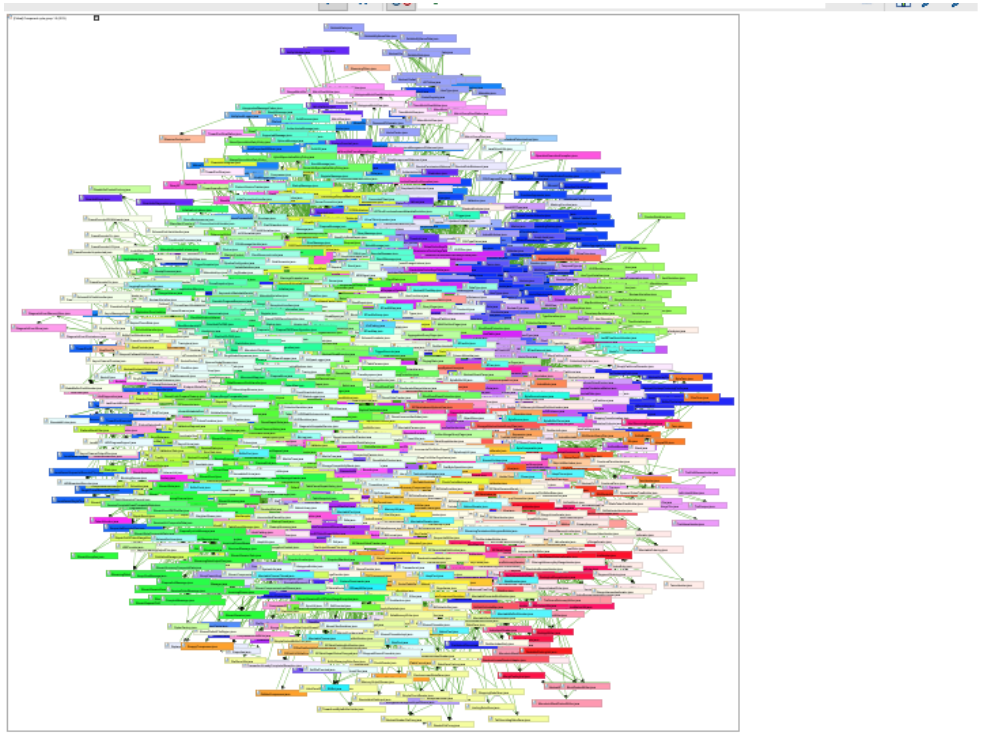
SR



V2

Universidad de La Sabana
FACULTAD DE INGENIERIA

Evolution from monolith



Properties | Parser Depende... | Parser Depende... | Markers | [Automated] ./C... | '[Critical] Comp...

Name [2,013]	Incoming	Outgoing	Incoming Interfac	Interface Score	Coupling Scor
ColumnFamilyStore.java	201	127	86	10,922	25,527
TableMetadata.java	367	48	109	5,232	17,616
DatabaseDescriptor.java	304	46	1	46	13,984
StorageService.java	76	169	1	169	12,844
SSTableReader.java	155	62	77	4,774	9,610
FBUtilities.java	245	34	0	0	8,330
ClusterMetadata.java	180	45	15	675	8,100
Verb.java	61	86	0	0	5,246
AbstractType.java	273	18	232	4,176	4,914
Keyspace.java	108	42	31	1,302	4,536
MessagingService.java	119	36	1	36	4,284
ColumnMetadata.java	152	28	62	1,736	4,256
ReadCommand.java	53	79	48	3,792	4,187
DecoratedKey.java	279	15	264	3,960	4,185
SystemKeyspace.java	58	71	0	0	4,118
PartitionUpdate.java	70	53	47	2,491	3,710
CassandraMetricsRegistry.java	61	55	1	55	3,355
ClientState.java	122	27	98	2,646	3,294
ActiveRepairService.java	37	77	3	231	2,849

Why Should a Company Decide to Build an Evolutionary Architecture?

Predictable versus evolvable: The Innovators Dilemma (companies in well-established markets are likely to fail as more agile startups address the changing ecosystem better).

Scale: No more Band-Aids, focus on decoupling

Advanced business capabilities: (A/B Testing) → As variation, Cycle time as a business metric

Adaptation versus evolution: Adaptation with Tech Debt on Feature Toggles vs Fundamental Change

Why Should a Company Decide Not to Build an Evolutionary Architecture?

Can't evolve a Big Ball of Mud: No feasibility

Other architectural characteristics dominate: Other QA instead of evolutionarily

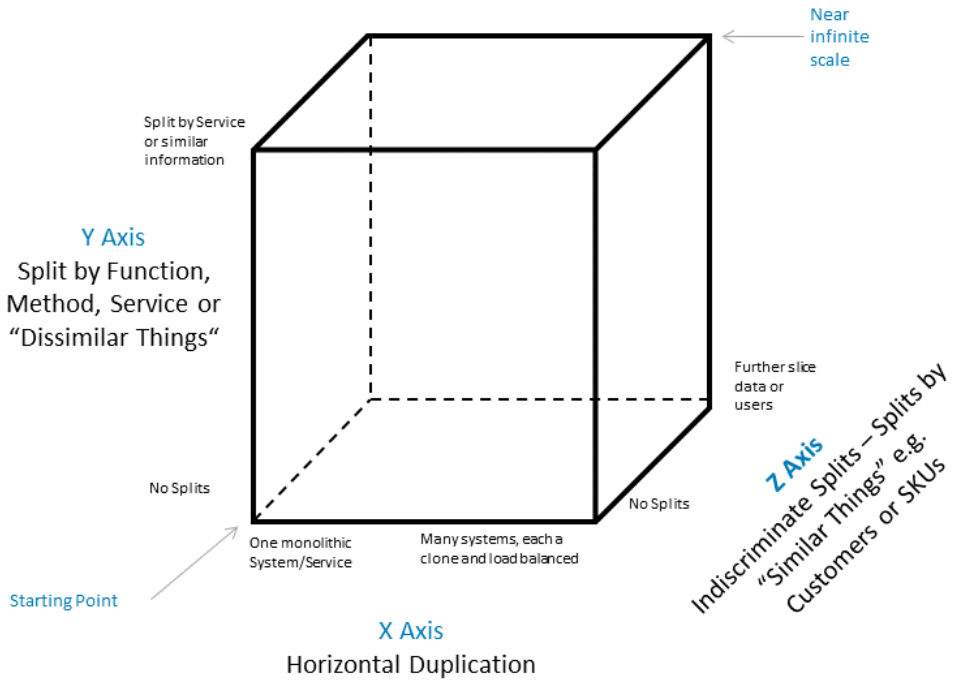
Sacrificial architecture: MVP

Planning on closing the business soon

Link and [BEAr, Ch9]

Scale Cube

AKF Scale Cube

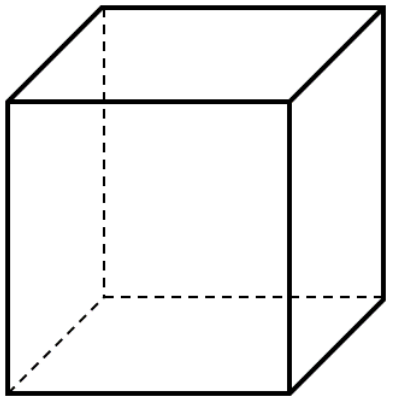


AKF Scale Cube

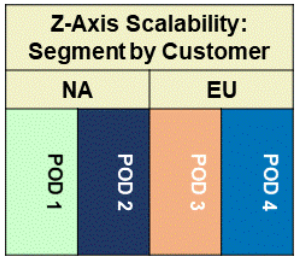
AKF Scale Cube

AKF Scale Cube

Pros	
Transaction Scalability	Scales transactions well
Latency	Reduces response times
Cache	Increases cache hit rate
Availability	Can have fault isolation



Cons	
Difficult	Can take time to architect
Complexity	More things to manage



X: Redundancy/HA → Previous subject (Cloud Services)
Y: Divide application → Microservices (This module)
Z: Divide data → Distributed Services (Next module)

In addition, you can find original at [AFK Partners]

Universidad de La Sabana
FACULTAD DE INGENIERIA

Features/Definition



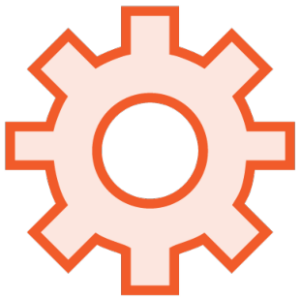
Micro

- Big or small
- No universal measure
- "Does one thing“
- Scope of functionalities
- Bounded context
- Identify sub-domains

…ervice architectural style is an …to developing a single application …uite of small services, each running in …own process and communicating with …ghtweight mechanisms.”

James Lewis and Martin Fowler, Thoughtworks

What are the identifiable QA which motivate microservices ?



Service

- Independe…mponent
- Interope…
- Mess…unication
- SC…

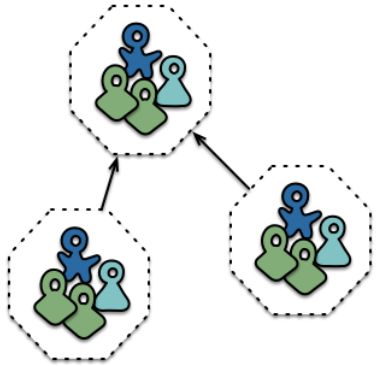
“Microservices are small, autonomous services that work together.”

Sam Newman, Thoughtworks

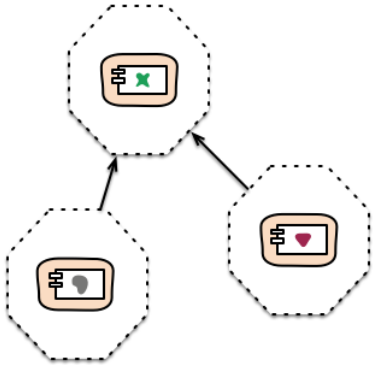
Universidad de La Sabana
FACULTAD DE INGENIERIA

Features/Definition

Componentization via Services
Organized around Business Capabilities
Products not Projects
Smart endpoints and dumb pipes
Decentralized Governance
Decentralized Data Management
Infrastructure Automation
Design for failure
Evolutionary Design

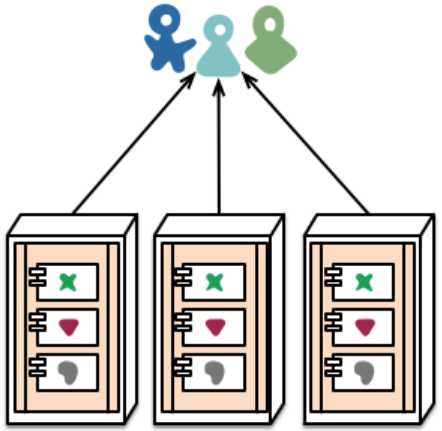


Cross-functional teams...

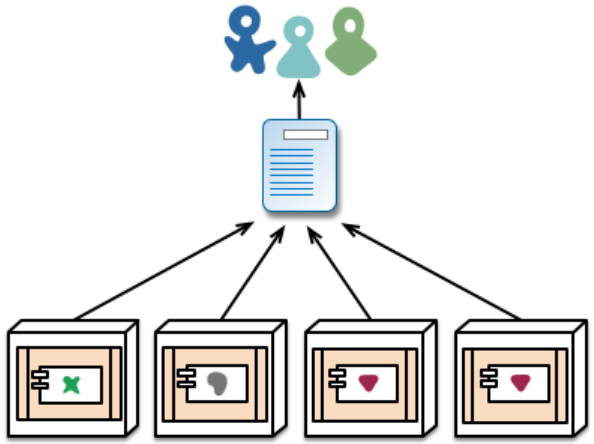


... organised around capabilities
Because Conway's Law

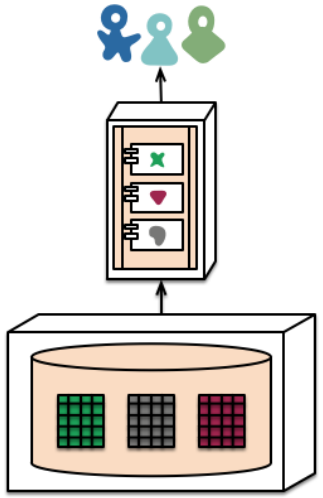
Divided:
- Technical
- Organizational
- Architectural



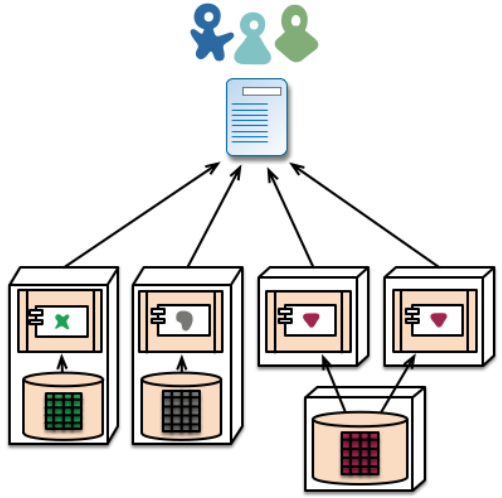
monolith - multiple modules in the same process



microservices - modules running in different processes



monolith - single database

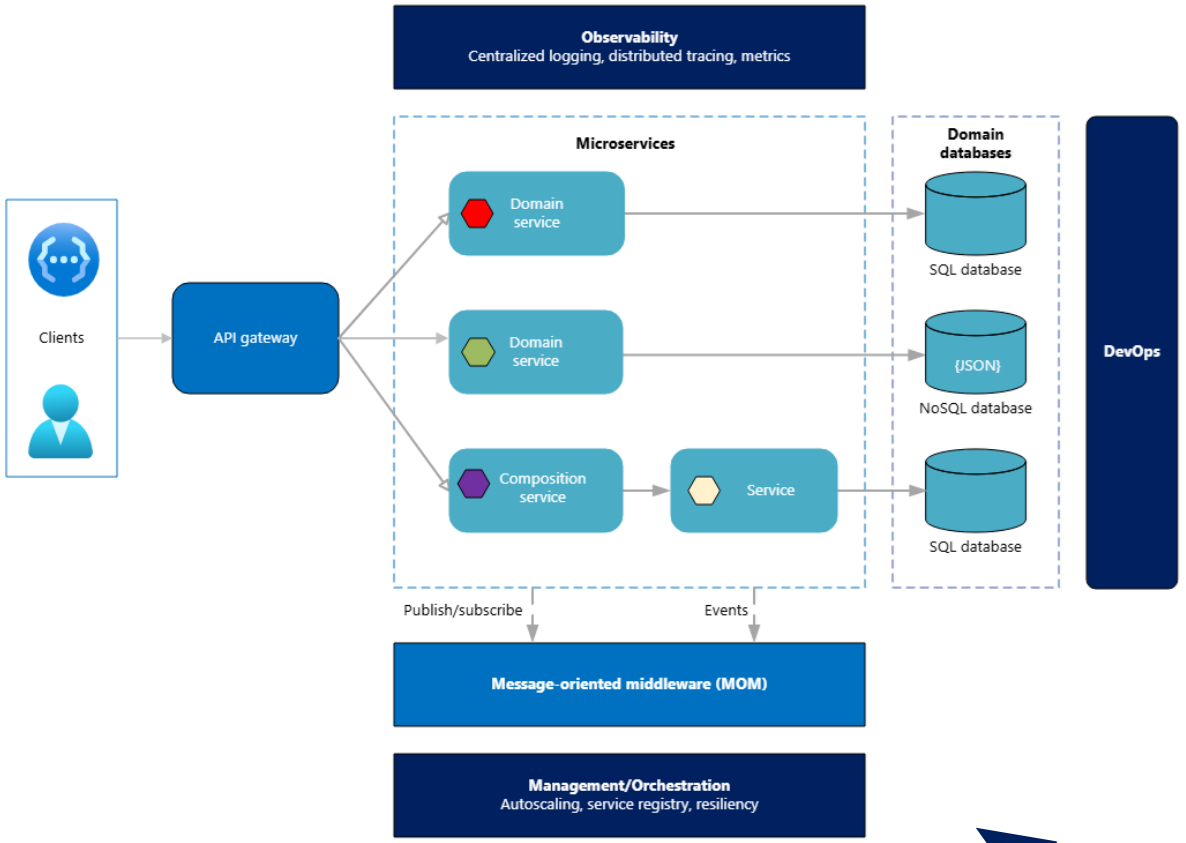


microservices - application databases

[MFMs]

Universidad de La Sabana
FACULTAD DE INGENIERIA

Definition comparison



Microservices are small, independent, and loosely coupled components that a single small team of developers can write and maintain.

Components

- Management or orchestration (Control Plane instead of Orchestration vs Choreography)
- API gateway
- Message-oriented middleware (Kafka o Service Bus)
- Observability
- Data management

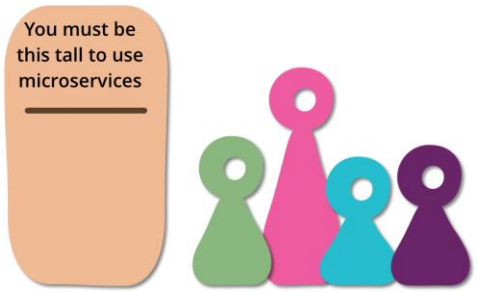
Benefits

Agility; Small, focused teams; Small code base; Mix of technologies; Fault isolation; Scalability; Data isolation

Challenges

- Complexity
- Development and testing
- Lack of governance
- Network congestion and latency
- Data integrity
- Management: CI/CD Pipelines
- Versioning
- Skill set

Microsoft Azure

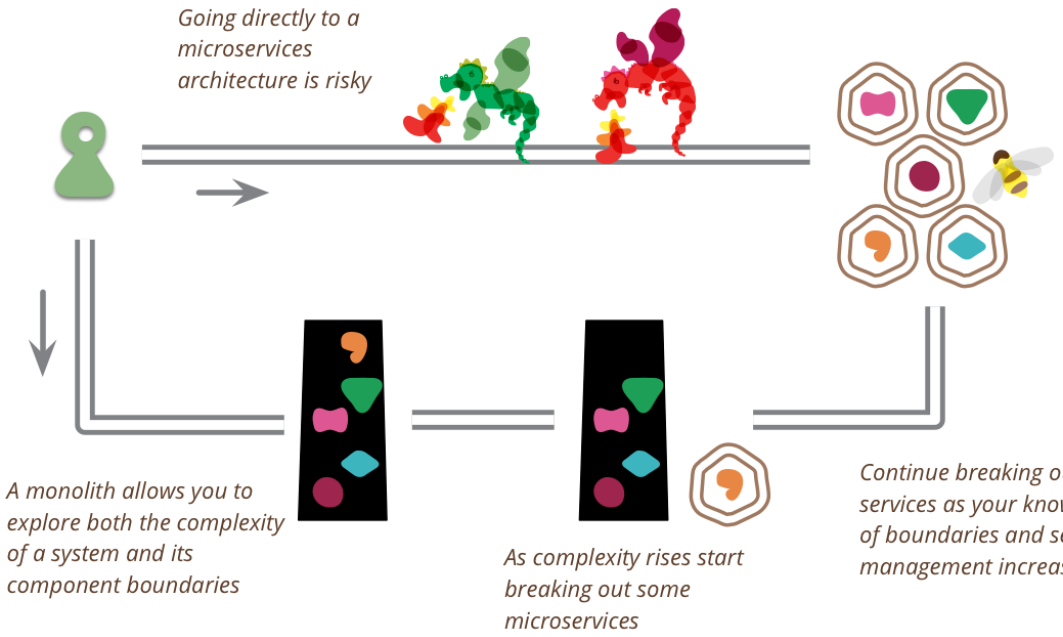


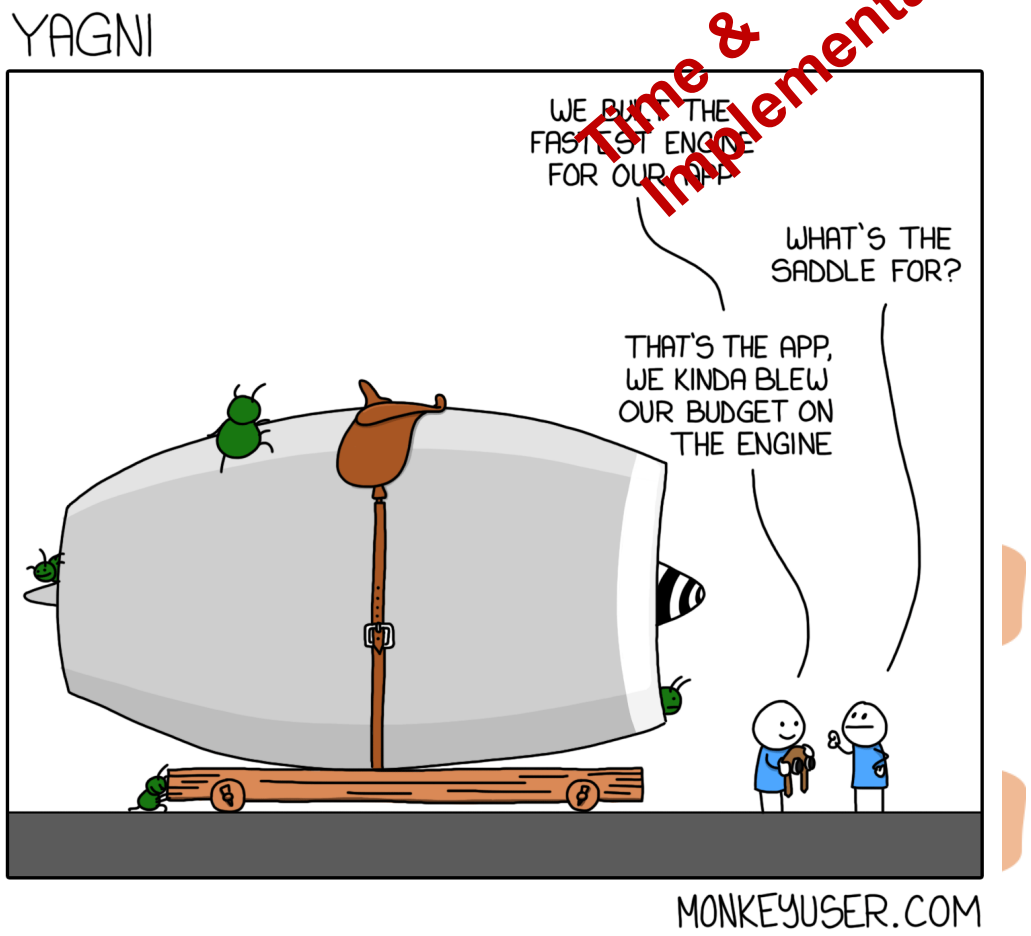
MF' Prerequisites
Rapid provisioning: Cloud
Basic Monitoring: Monitoring tech/biz issues
Rapid application deployment: CI/CD Pipelines

[AzDA] and link

Universidad de La Sabana
FACULTAD DE INGENIERIA

Monolith-First - YAGNI





Time & Implementation

Squad/Team/Tribe/Cell Maturity and Well-Design Thinking

Designing a good (monolithic or microservice) architecture with Assemblage

There are two key issues that you must address. The first issue is whether to use the monolithic or microservice architecture.

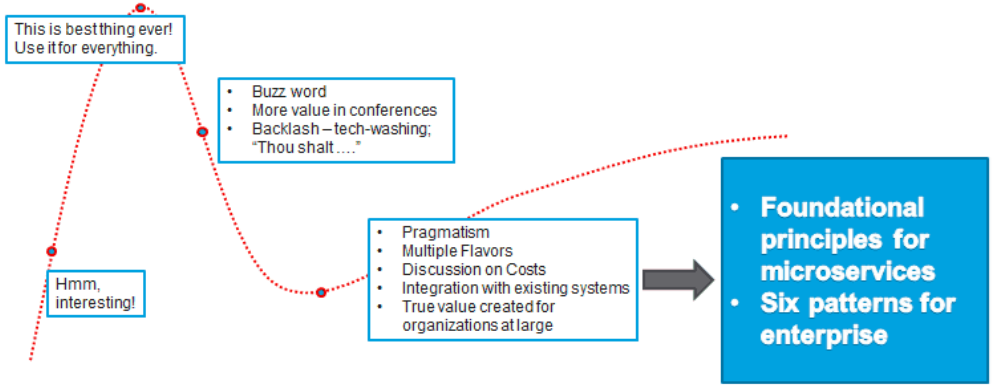
Golden bullet ? → Capacity and Maturity
Req. Timeline (Market Change?)
Dummy interfaces (API GW, LB) → Cost of Carry
API Design →Business Expert

[MFMf, MFYa and MsAP]

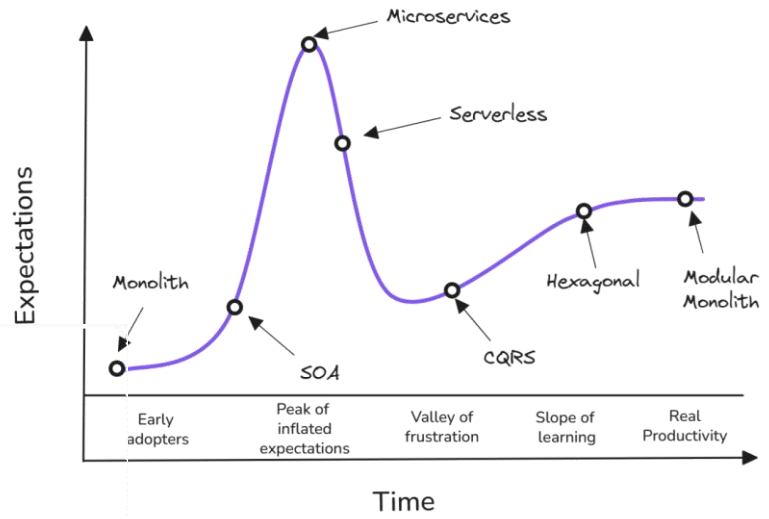
Universidad de La Sabana
FACULTAD DE INGENIERIA

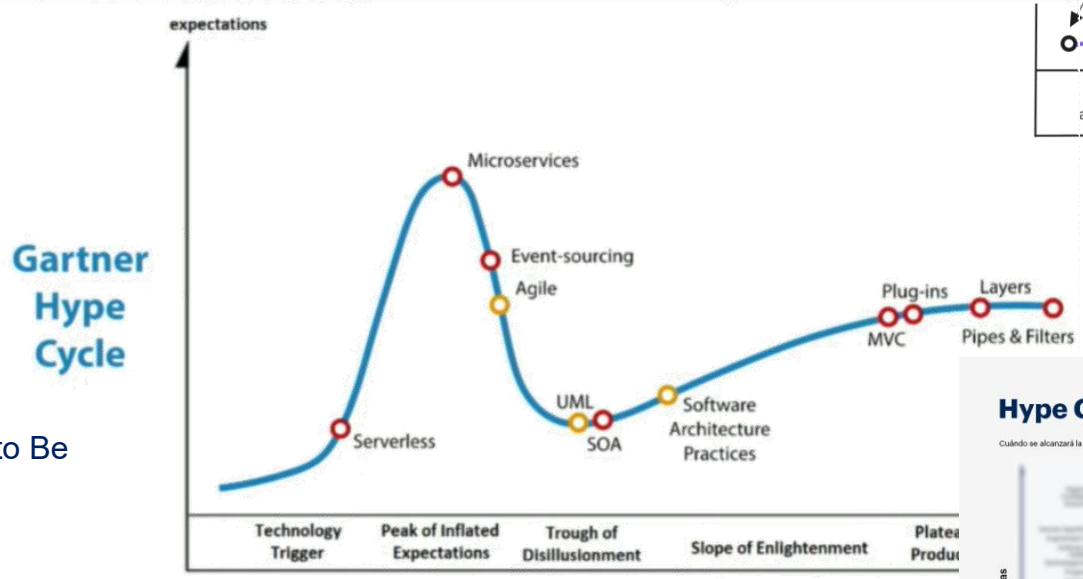
Software Architecture Hype Cycle

"Coolness factor" of emergent technologies

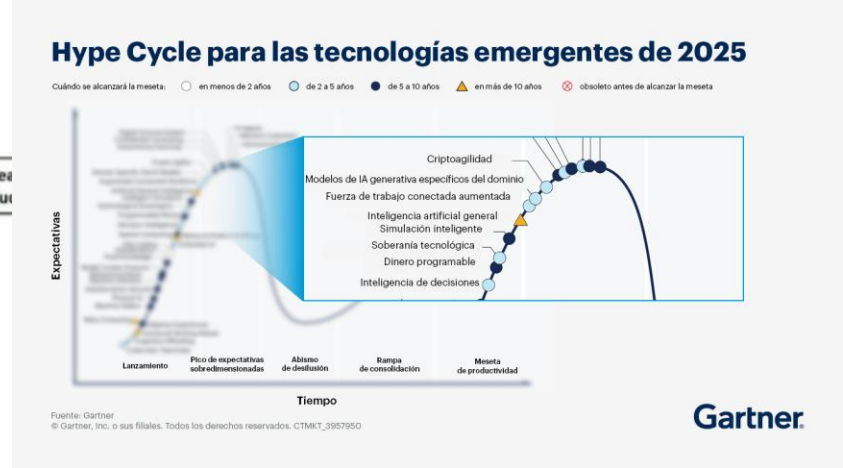


All contents © MuleSoft Inc.





TechWorld WithMilan



[2007] Mulesoft Blog. Are Microservices Too Cool to Be Useful?. Sandeep Singh Kohli
[2018] NDC Conf. Deliberate Architecture. Robert Smallshire
[2026] TechWorld with Milan, Software Arch Hype Cycle. Dr. Milan Milanovic

Example from
[2025] Gartner, Emergent Technologies on 2025.

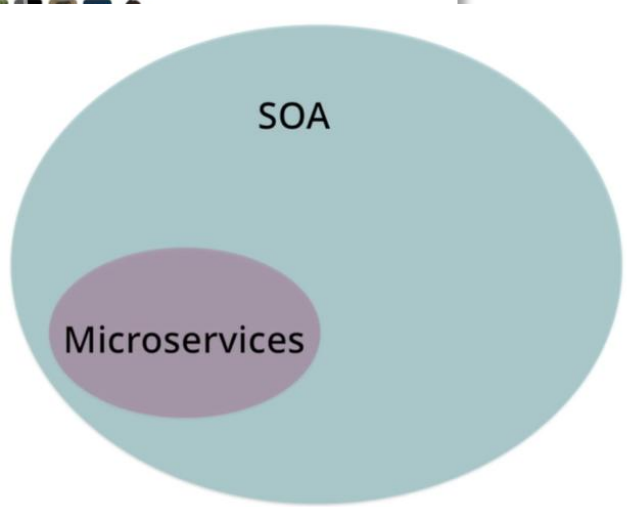
Links

Universidad de La Sabana
FACULTAD DE INGENIERIA

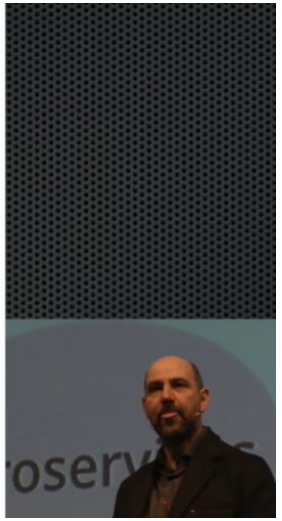
SOA, SBA, MSA, Functions and so on

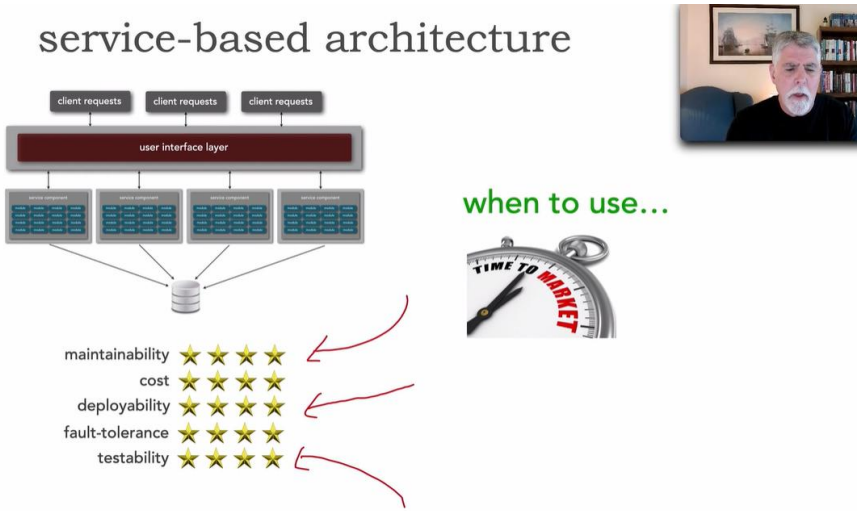






Dimension	Service-Oriented (SOA)	Service-Based (SBA)	Microservices Arch (MSA)
Primary Scope	Enterprise Integration	App-level Modularity	Fine-grained Agility & Scale
Service Size	Coarse (Large Macro-services)	Medium (Domain Modules)	Fine (Single Responsibility)
Database Strategy	Shared Enterprise Database	Commonly Shared Instance	strictly Database-per-Service
Integration Middleware	Centralized ESB (Smart Pipes)	Lightweight Gateway/Broker	Decoupled (Dumb Pipes)
Deployment Dependency	High Coordinated Coupling	Moderate	Completely Autonomous



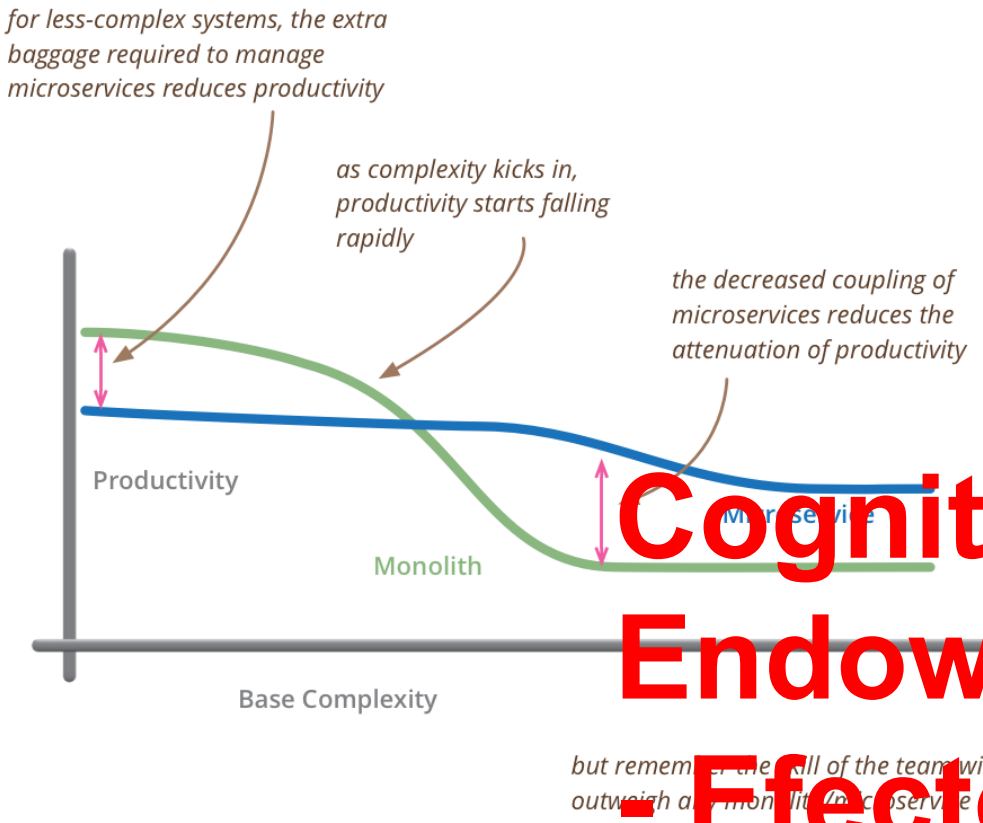


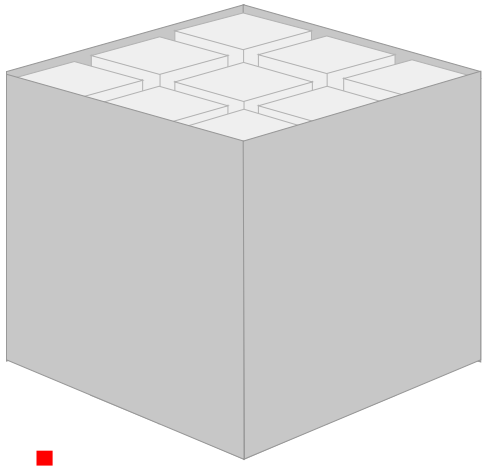
See sidebar on [MFMs] too

Links

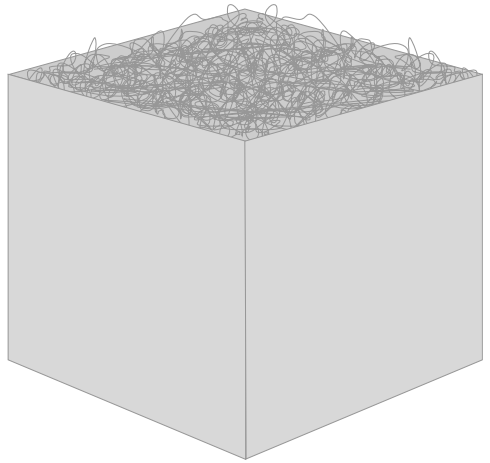
Universidad de La Sabana
FACULTAD DE INGENIERIA

What is it mean ?





Hope vs. Reality



Cognitive bias: Endowment Effect - Efecto de Dotación

- I hope you aren't a presales engineer: goal on sales or specific services
- Your focus is real productivity and reduced time-to-market
- What are the factors to choose?: tech (requisites, current services), non-tech (team maturity, management, egos)

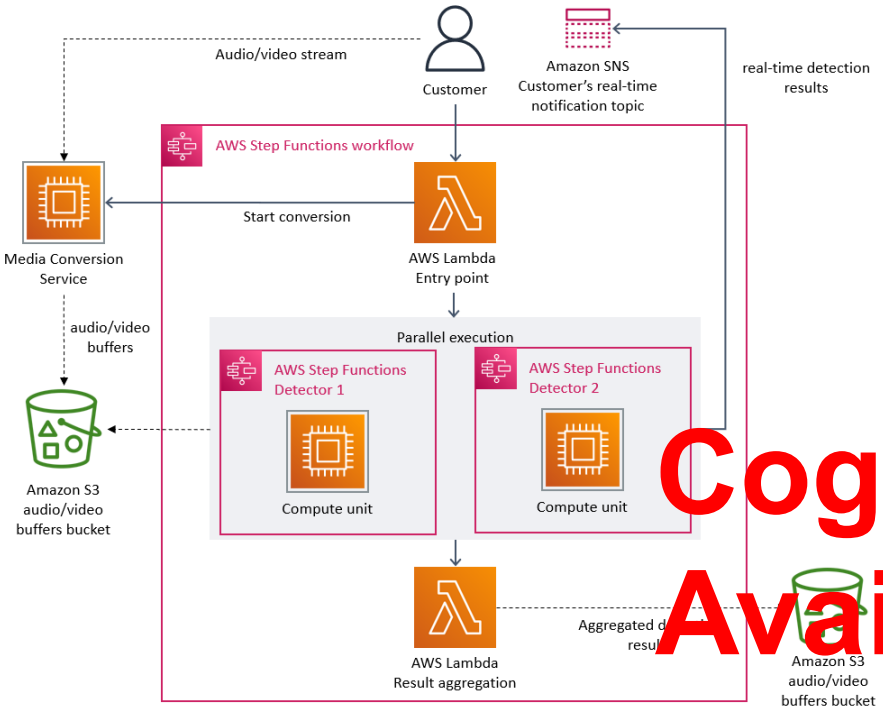
- Greenfield vs Brownfield projects: Starting point.
- Monoliths breed irreversible tight coupling
- Hard boundaries protect system architecture
- The "refactoring shift" (Small vs. Large)

Links

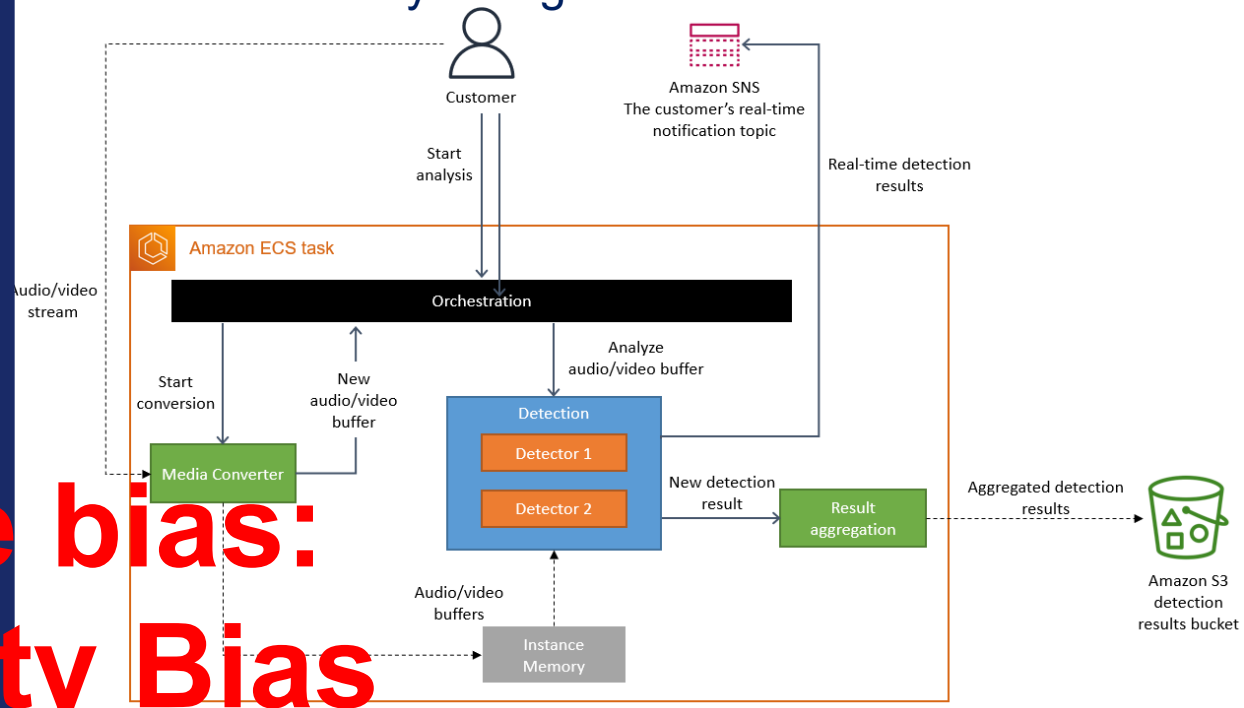
Universidad de La Sabana
FACULTAD DE INGENIERIA

Clever example of overarchitecting

Original Design: Microservices



Evolutionary Design: Monolith



Problem:
Review defect after Media Conversion
Initial Review:

- Orchestrate using Step Functions
- Lambda concurrency
- Data Exchange using S3

>> Initial MVP << Concurrency and Scalability

Solution:
Orchestration
Provisioning compute services
Local Storage

Cognitive bias: Availability Bias - Sesgo de disponibilidad

[PVRe] and [PVBa], GitHub

Universidad de La Sabana
FACULTAD DE INGENIERIA

12 Factor Methodology

THE TWELVE FACTORS

I. Codebase
One codebase tracked in revision control, many deploys

II. Dependencies
Explicitly declare and isolate dependencies

III. Config
Store config in the environment

IV. Backing services
Treat backing services as attached resources

V. Build, release, run
Strictly separate build and run stages

VI. Processes
Execute the app as one or more stateless processes

VII. Port binding
Export services via port binding

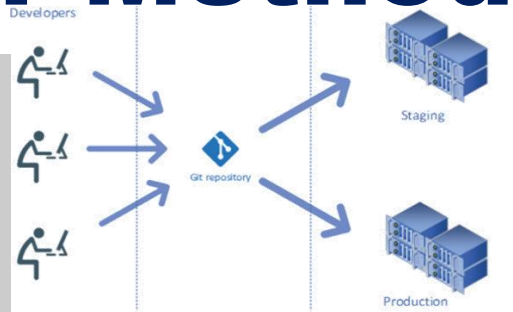
VIII. Concurrency
Scale out via the process model

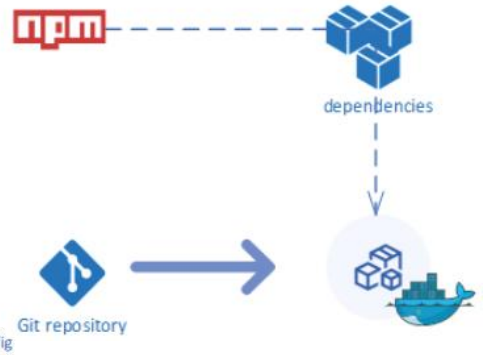
IX. Disposability
Maximize robustness with fast startup and graceful shutdown

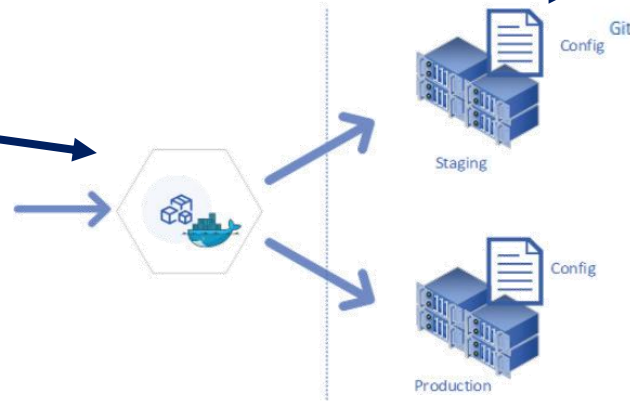
X. Dev/prod parity
Keep development, staging, and production as similar as possible

XI. Logs
Treat logs as event streams

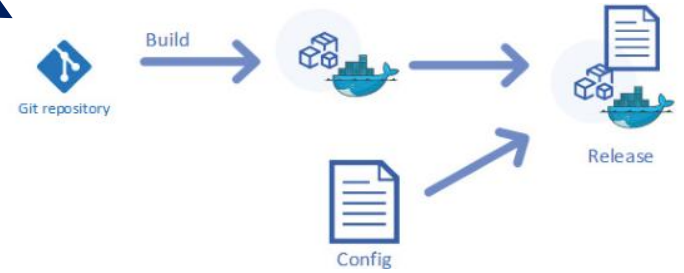
XII. Admin processes
Run admin/management tasks as one-off processes

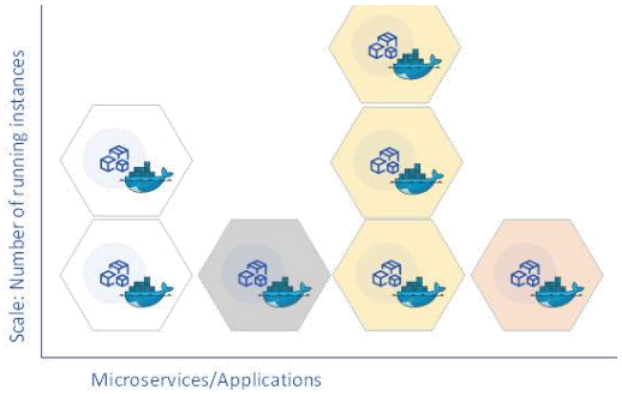












Beyond of…
API First
Telemetry
Security

Links and [EmMD]

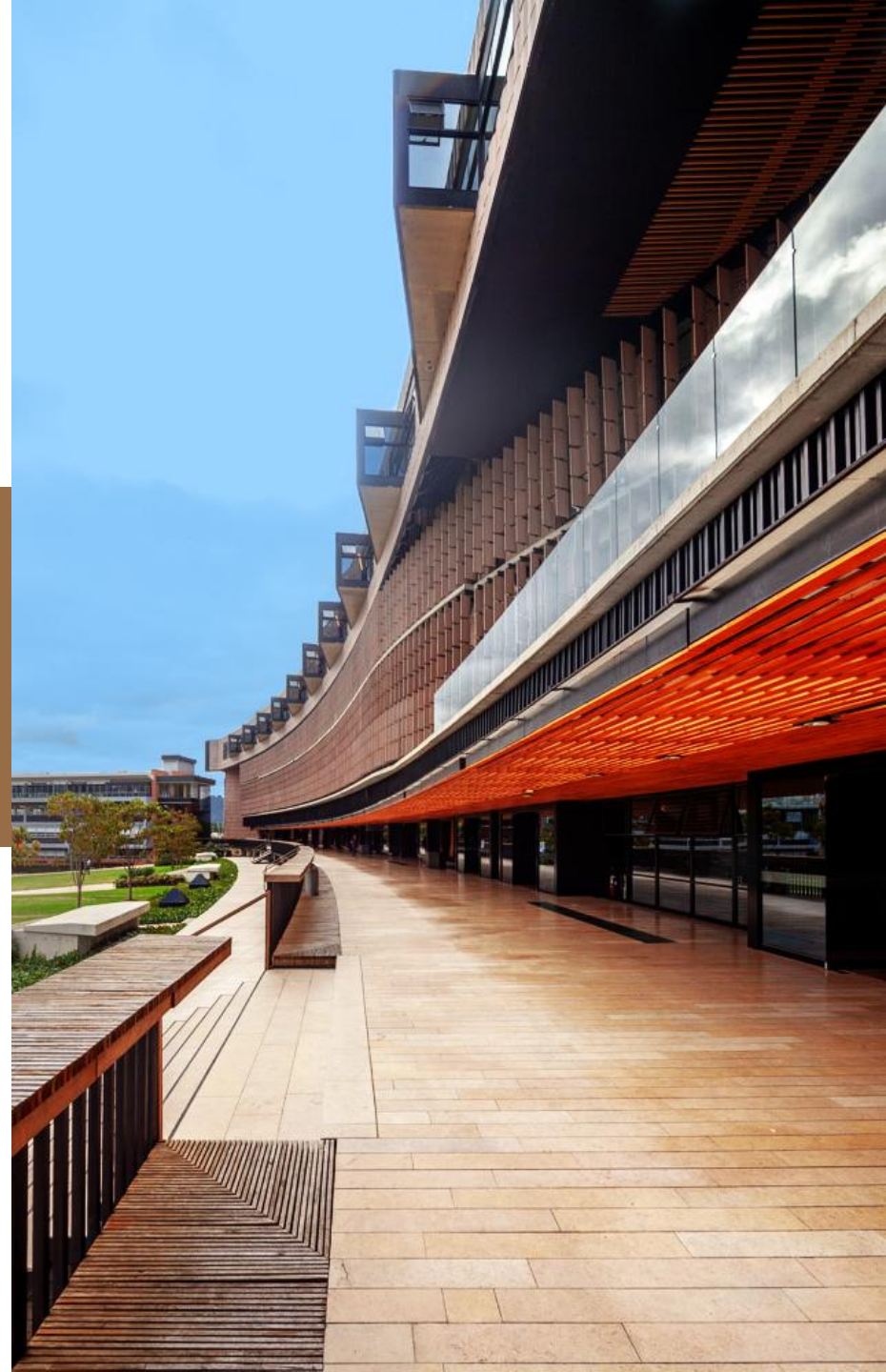
Universidad de La Sabana

Design Techniques: DDD

Techniques then, my experience

Universidad de La Sabana

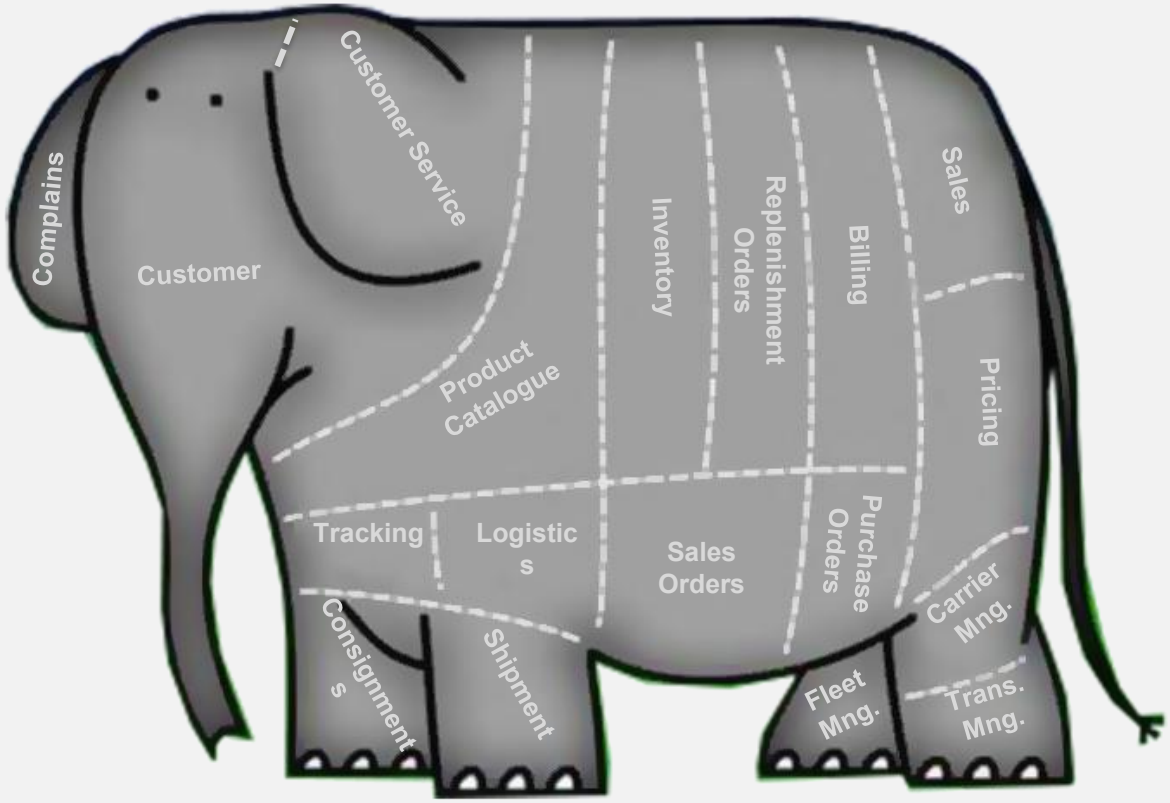
FACULTAD DE INGENIERÍA



Universidad de La Sabana
FACULTAD DE INGENIERIA

How to eat the elephant?

One piece a the time!



Understand the problem. Slice and dice your elephant by defining boundaries in the business capabilities. Modernise one piece at the time. Starting small

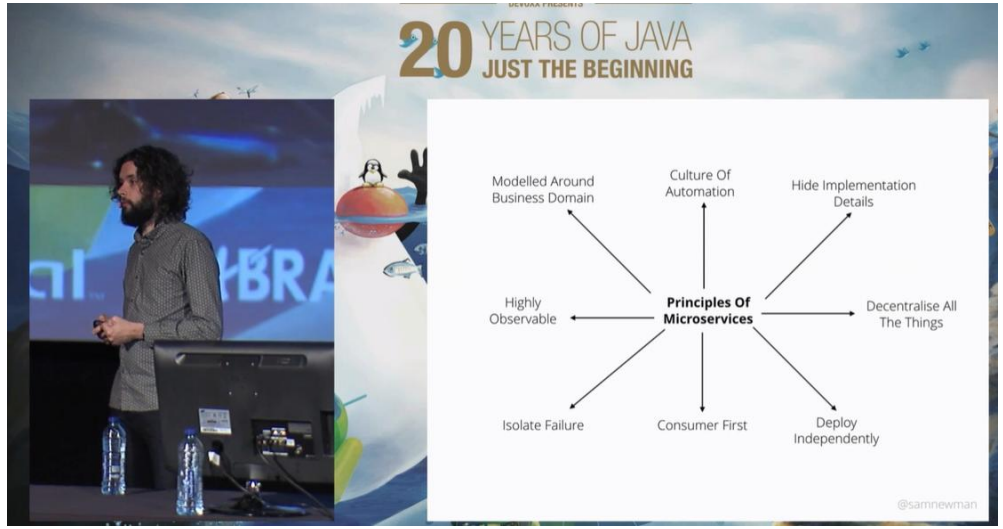
Link

Links

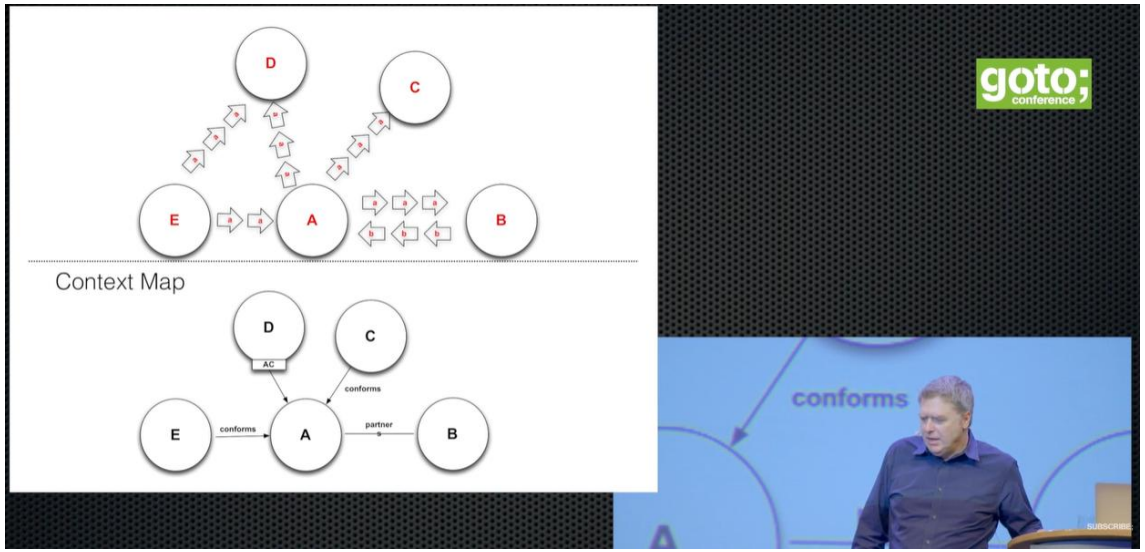
Universidad de La Sabana
FACULTAD DE INGENIERIA

DDD & Microservices

What are their purposes ?
What are the differences ?
Is enough to know at detail ?
Real application in a Colombia culture?



Principles Of Microservices by Sam Newman, DevOox Belgium 2015



DDD & Microservices: At Last, Some Boundaries! • Eric Evans • GOTO 2015

"While Domain-Driven Design provides many technical benefits, such as maintainability, it should be applied only to **complex domains** where the model and the linguistic processes **provide clear benefits** in the **communication of complex information**, and in the formulation of a **common understanding of the domain**."

Links

Universidad de La Sabana
FACULTAD DE INGENIERÍA

Pattern Language



Domain Language

Tackling Complexity in the Heart of Software

HOME TRAINING ELEARNING DDD RESOURCES UPCOMING EVENTS ... ABOUT US

Read Eric's Blog ...

Articles »

About Domain Language

We are a small consultancy focused on Domain-Driven Design (DDD) and led by Eric Evans, who wrote the first book on DDD. Our current focus is on helping organizations thoughtfully integrate AI technologies like LLMs into domain-rich systems while preserving the design integrity that delivers business value.

Check out DDD eLearn!

Our video-based course on Domain-Driven Design (DDD) is just over 5 hours of tightly edited video. This self-guided course focuses on the deep concepts of DDD,

Search ...

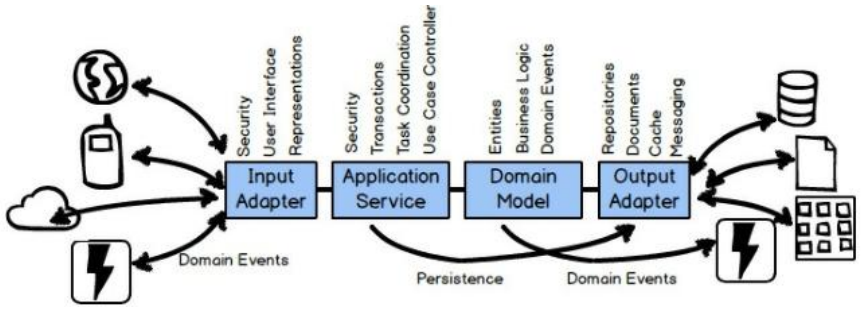
CONTACT US

Send us a message! We'll respond within two business days.

VIRTUAL DDD

A community-driven platform for people who want to get more in-depth knowledge of Domain-Driven Design and solving complex business problems.

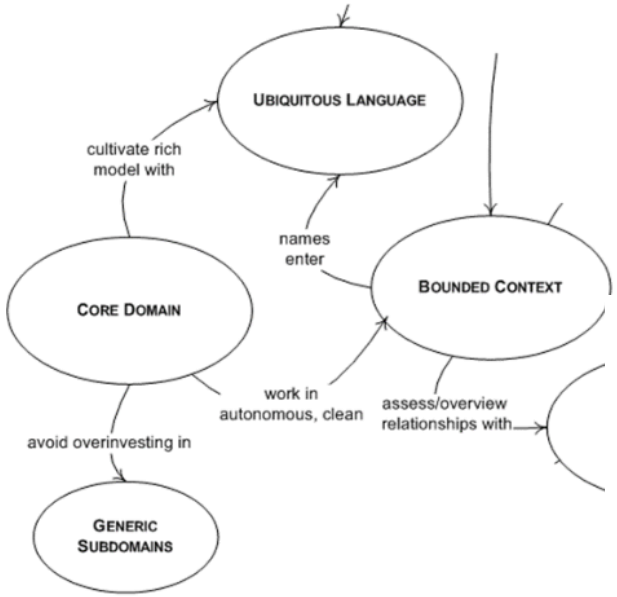
If you don't live near an active Domain-Driven Design meetup, or just want to get more knowledge of DDD, please join



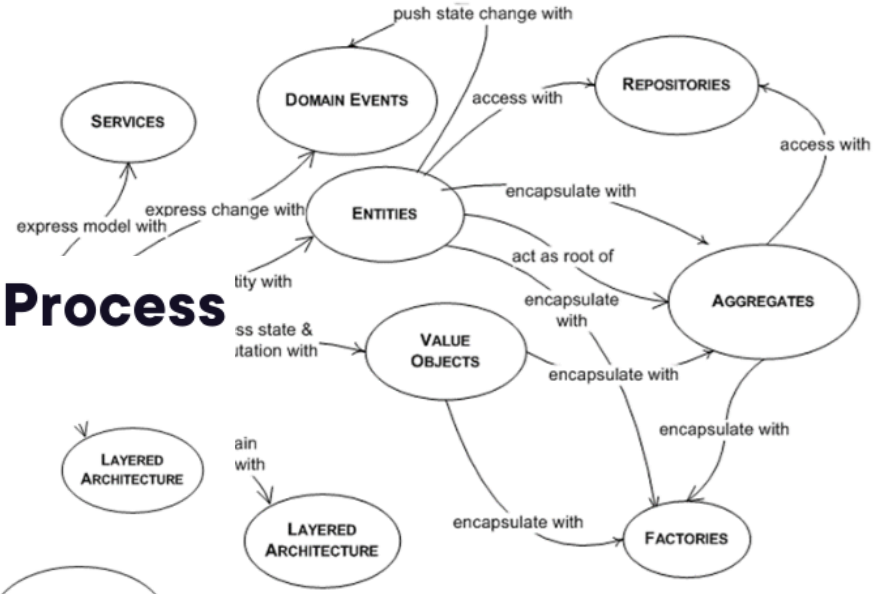
Universidad de La Sabana
FACULTAD DE INGENIERIA

Drill down

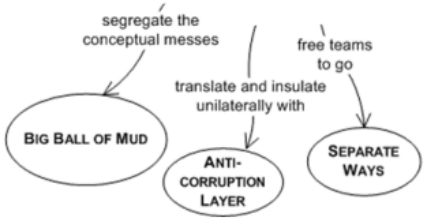
DDD Mind Map: Modeling



DDD Mind Map: Software Implementation



DDD Mind Map: Development Process



Links

Universidad de La Sabana
FACULTAD DE INGENIERIA

Definitions

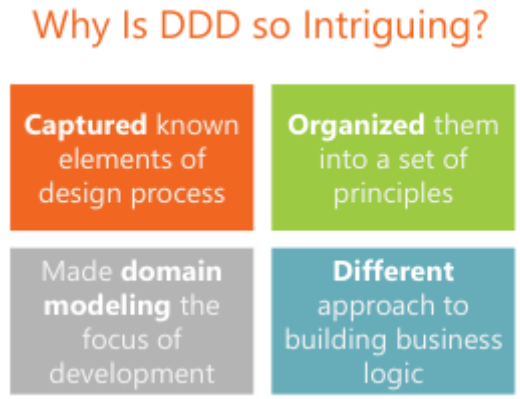
domain
A sphere of knowledge, influence, or activity. The subject area to which the user applies a program is the domain of the software.

model
A system of abstractions that describes selected aspects of a domain and can be used to solve problems related to that domain.

ubiquitous language
A language structured around the domain model and used by all team members within a bounded context to connect all the activities of the team with the software.

context
The setting in which a word or statement appears that determines its meaning. Statements about a model can only be understood in a context.

bounded context
A description of a boundary (typically a subsystem, or the work of a particular team) within which a particular model is defined and applicable.



Universidad de La Sabana
FACULTAD DE INGENIERIA

Ubiquitous Language

For instance, shipping company uses a device to track its vehicles. Name it:

- Tag
- Beacon
- Tracker
- Locator
- Telemetry Unit (More Info)

Geek confusion:
Class (soft. eng.) != Class (from Training Class)
Entity (MS ORM) != Entity (from DDD)

Noun

Verb



Registered Customer

Redeem

Voucher

Order

Place

Pay

Ordered Items

- **Voucher** is the domain name.
- Synonyms like coupon or gift card are **not** allowed.

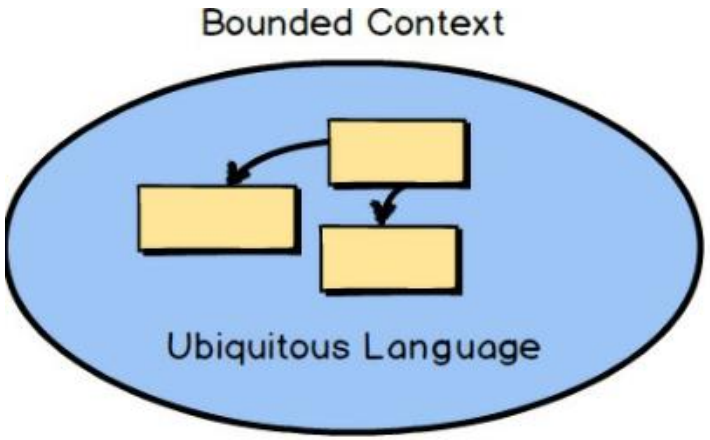
Link

Universidad de La Sabana
FACULTAD DE INGENIERIA

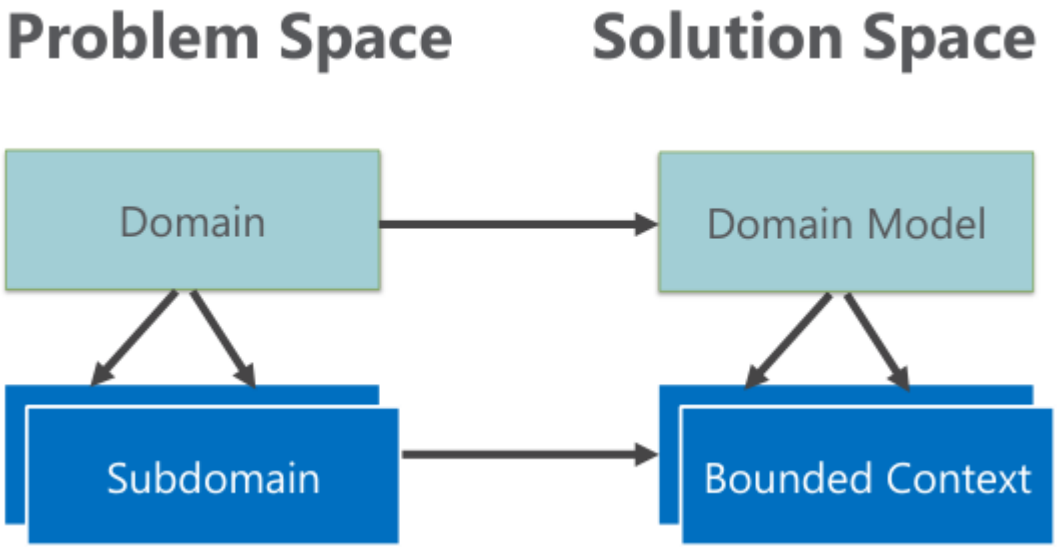
Bounded Contexts

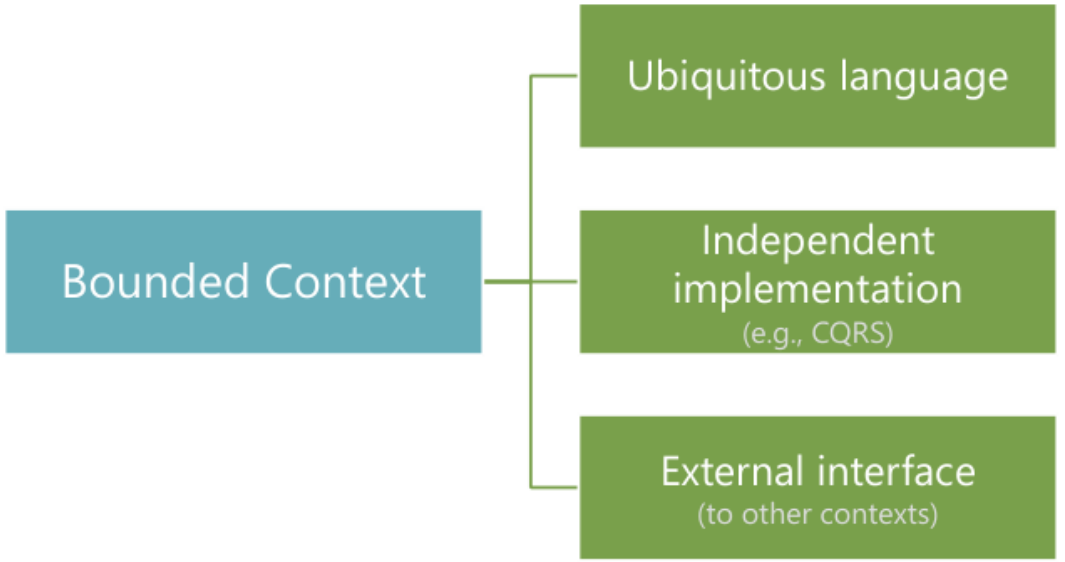
Multiple models are inevitable (….) → Abstractions, models, even solutions based on AI → confussion, BBM.

Model expressions, like any other phrase, only have meaning in context. → Explicitly define the context within which a model applies. Explicitly set boundaries in terms of {org, app, code, db}



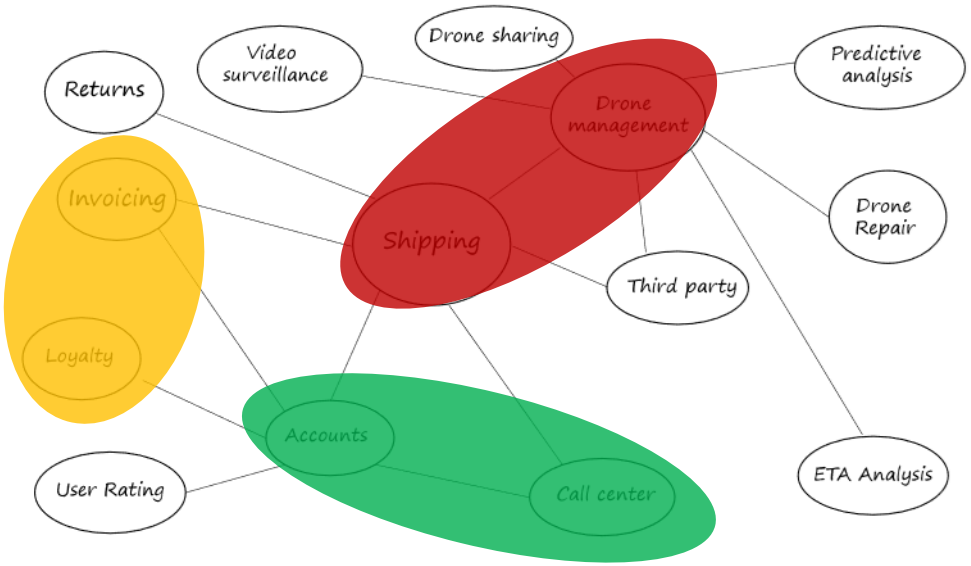
Language → Modeling → R:/ Isolation, Consistency, Deliverable

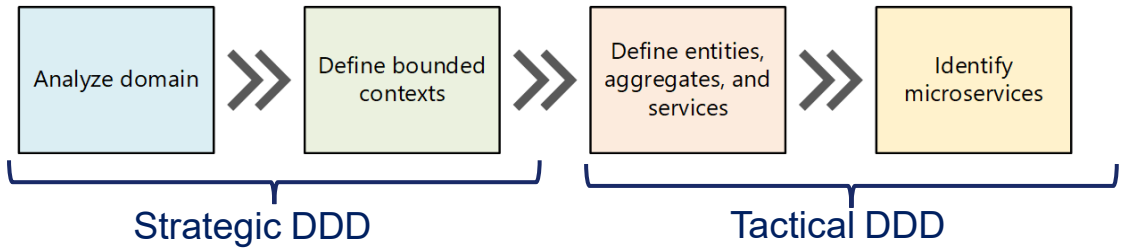




Universidad de La Sabana
FACULTAD DE INGENIERIA

P1: Analyze Domain





Analyze domain
Initial Sketch: Data objects, actors, process.
Identify your effort through:
Core {subdomain}: Your competitive advantage
Supporting {subdomain}: Enterprise/In-house solutions
Generic {subdomain}: Industry/IT solutions-> COTS

….Complexity, change freq (high, stable, low), DDD tactic (man, opt), effort

Universidad de La Sabana
FACULTAD DE INGENIERIA

A Talk with Eric Evans

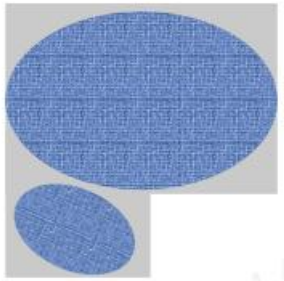
Problem Space vs. Solution Space



Problem Space
Bare floor of a room



Solution Space A
Wall-to-wall carpet
Same shape and size as the problem



Solution Space B
Area rugs
Clearly a different shape and size as the problem

It's not a simple task, even with experience.
Lack of clear boundary impedes the application of DDD ideas.
Bounded context is an essential ingredient.
Defining boundaries is the biggest stumbling block.

Working on a Common, Ubiquitous Language for the Appointment Scheduling Bounded Context

Working on a Common, Ubiquitous Language for the Appointment Scheduling Bounded Context

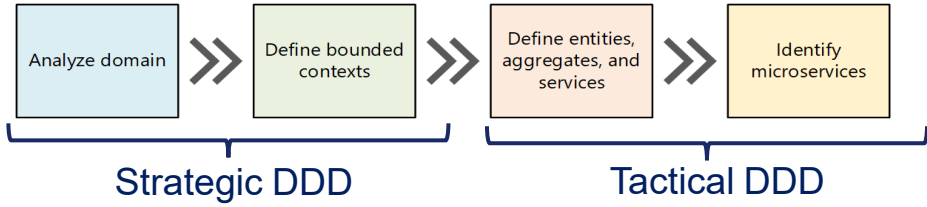
Clients	Person who: Makes appointments Pays the bills Brings pets to clinic
Patients	Animal for whom: Medical records are kept
Appointment	Scheduled time for: Surgeries Any type of regular office visit

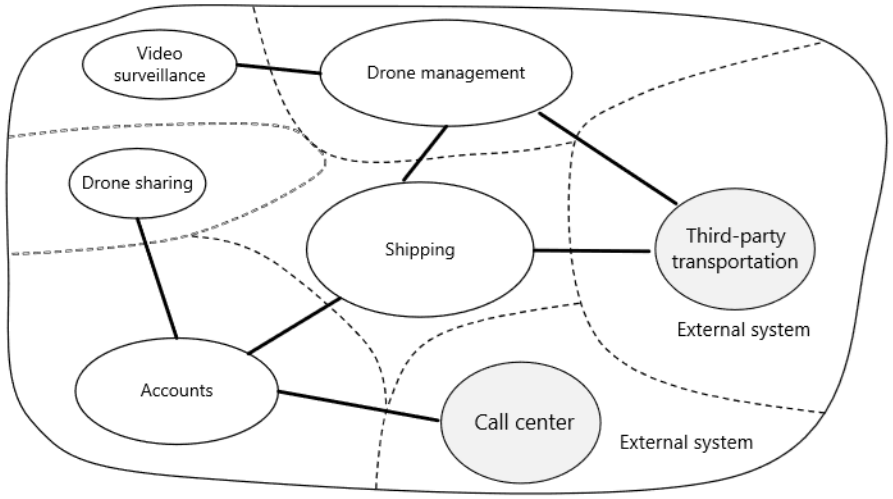
Problem solving

Tech task

Universidad de La Sabana
FACULTAD DE INGENIERIA

P2: Define bounded contexts

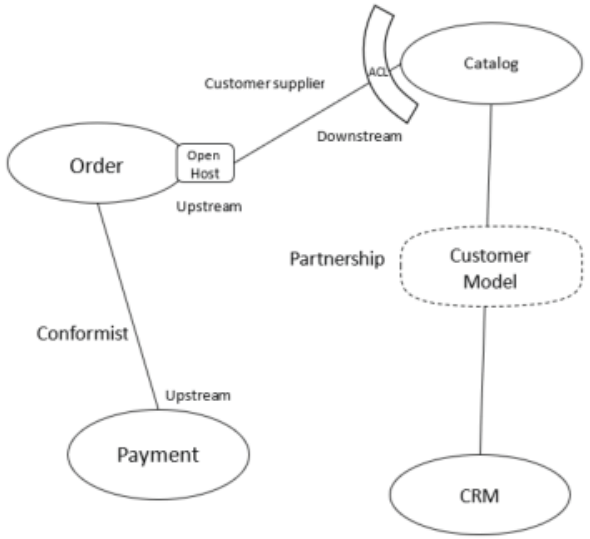




Define bounded contexts

Define context maps (relation):
{Upstream to Downstream}

- Customer/Supplier
- Conformist
- Partnership
- *Anticorruption Layer*
- *Shared Kernel*



Relationships

Conformist	• Downstream context depends on upstream context • No negotiation possible
Customer/Supplier	• Customer context depends on supplier context • Chance to raise concerns and have them addressed in some way
Partner	• Mutual dependency between the two contexts
Shared Kernel	• Shared model that can't be changed without consulting teams in charge of contexts that depend on it.
Anti-corruption Layer	• Additional layer giving the downstream context a fixed interface no matter what happens in the upstream context

Universidad de La Sabana
FACULTAD DE INGENIERIA

Conway's Law

Laws of Software Engineering

NEWSLETTER INFO BOOK POSTER

Organization

Software



Small distributed team



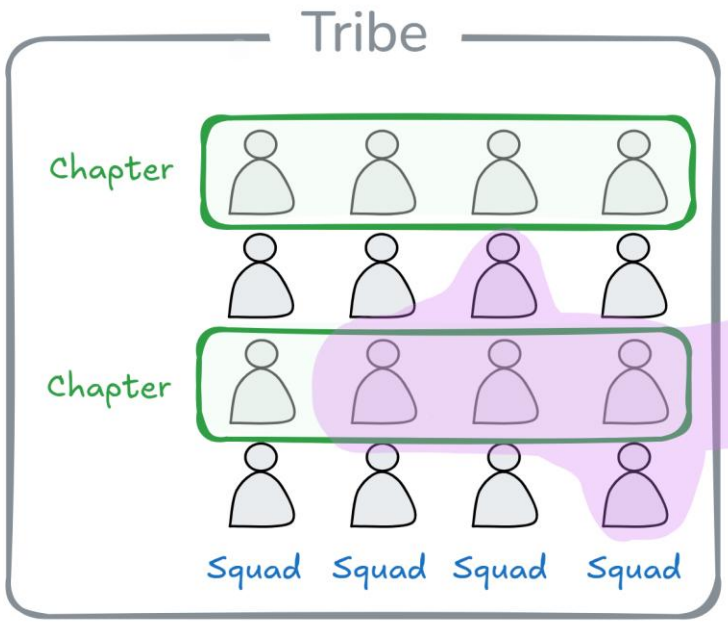


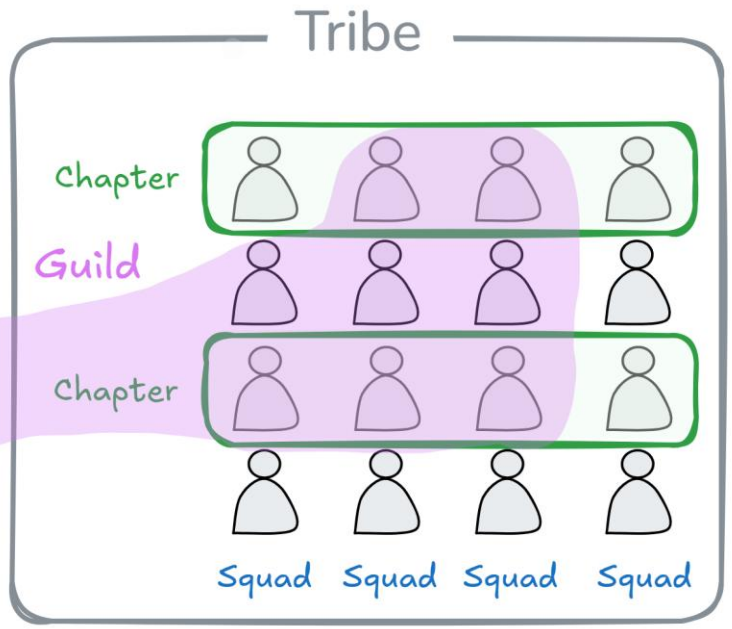


Large team





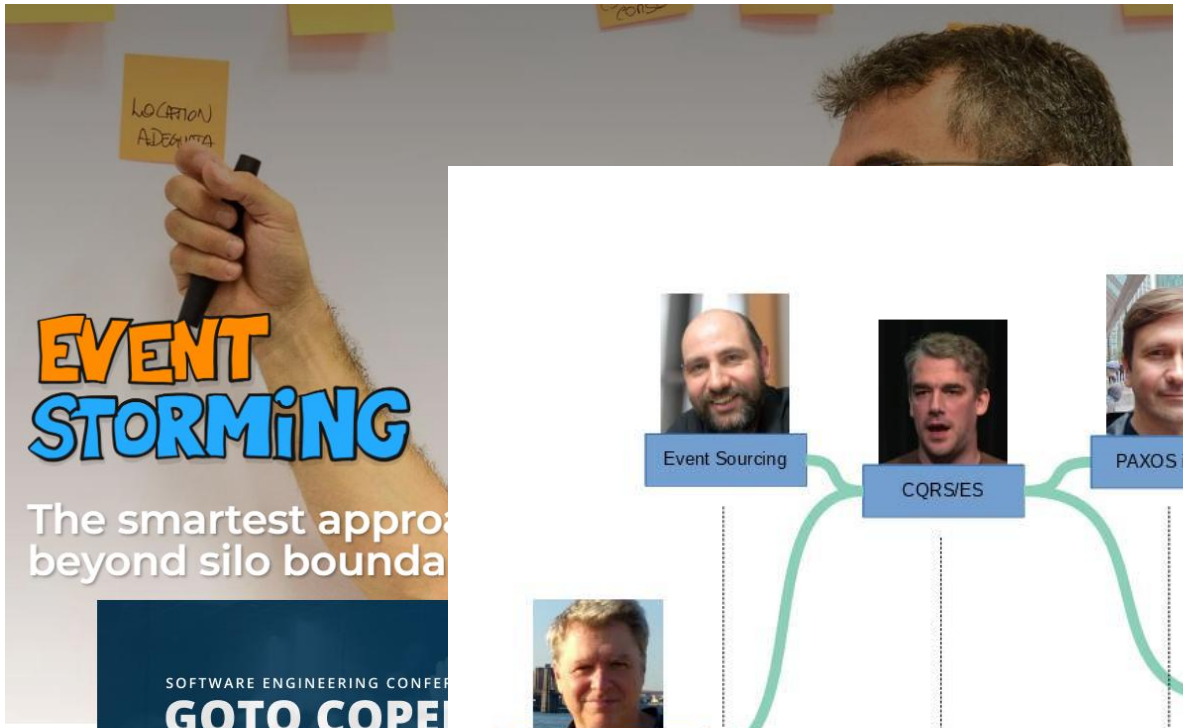




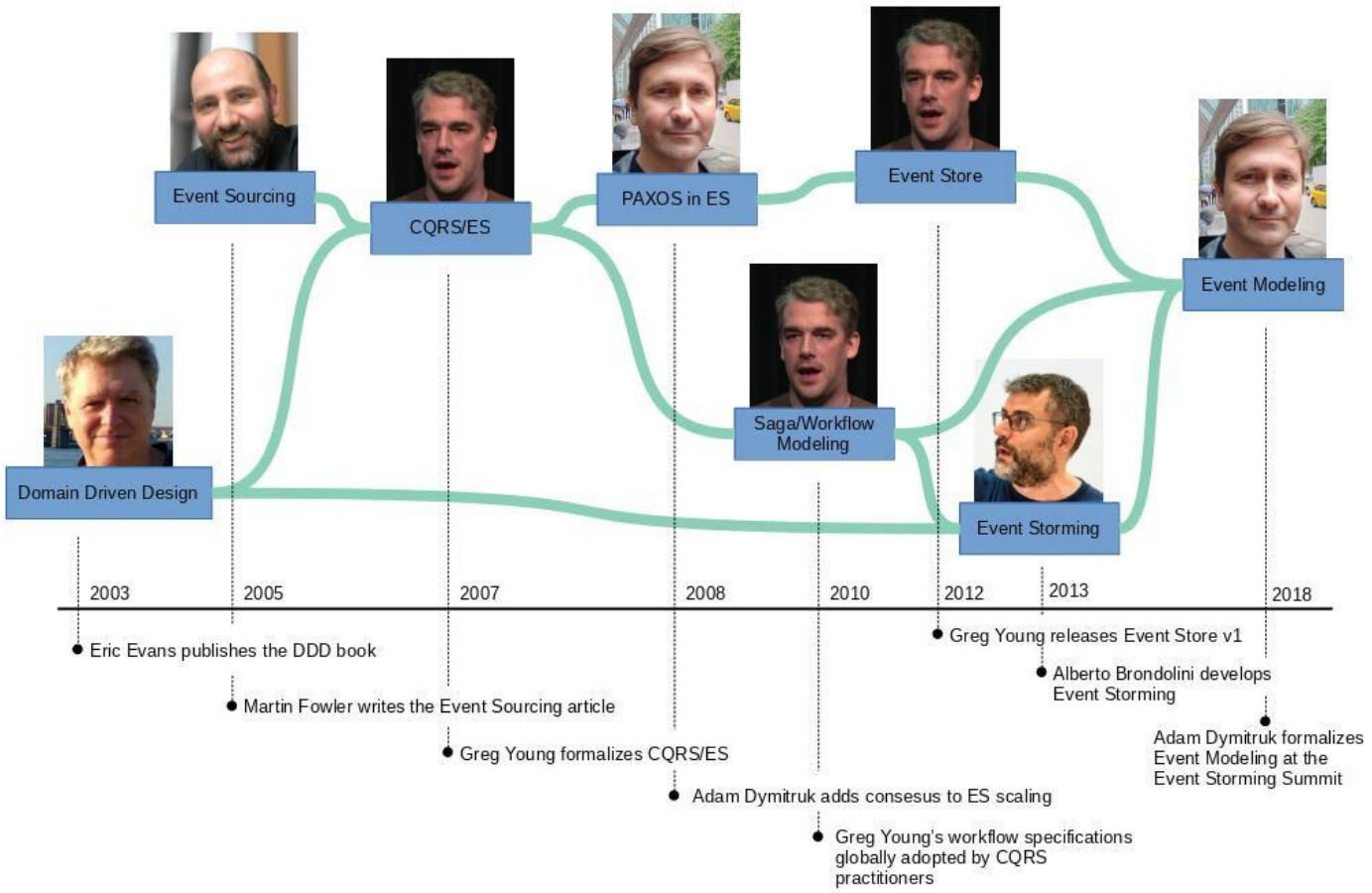
Links

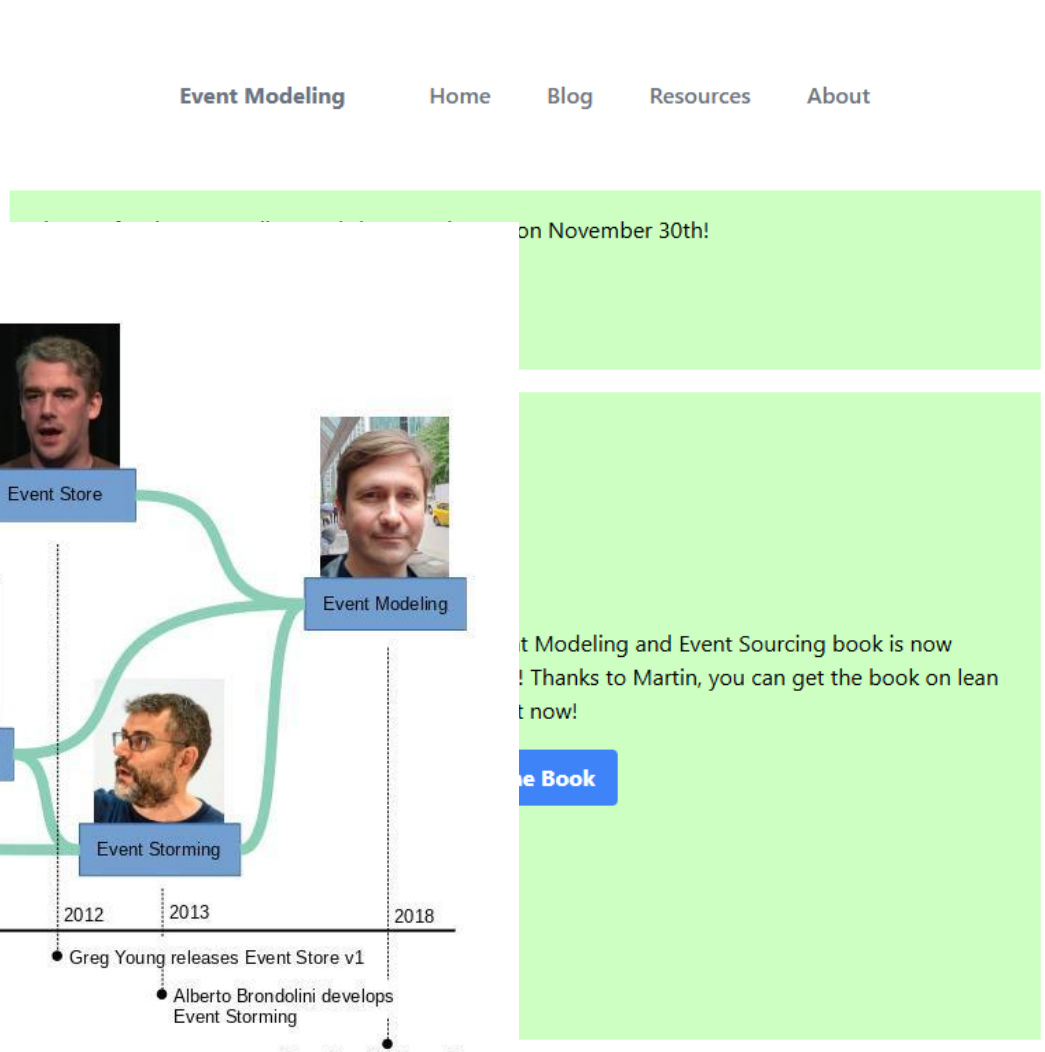
Universidad de La Sabana
FACULTAD DE INGENIERIA

Modeling



in colored sticky notes: orange f
purple for policy, hot pink for issu
the data needed by a user to ma
which are placed on a wide wall





Design Thinking ?

Universidad de La Sabana
FACULTAD DE INGENIERIA

Example of modelling

Notes from Our Conversation



Clients (people) schedule appointments for patients (pets)

Appointments may be either office visits or surgeries

Office visits may be an exam requiring a doctor, or a tech visit

Office visits depend on exam room availability

Surgeries depend on O/R and recovery space availability, and can involve different kinds of procedures

Different appointment types and procedures require different staff

"As software developers, we fail in two ways: We build the thing wrong, or We build the wrong thing."
Steve Smith

Distinct Data, Code, and Teams per Bounded Context?



Rarely see that level of separation in real world

You can introduce separation through namespaces, folders, and projects

While 100% is not achievable, strive for it

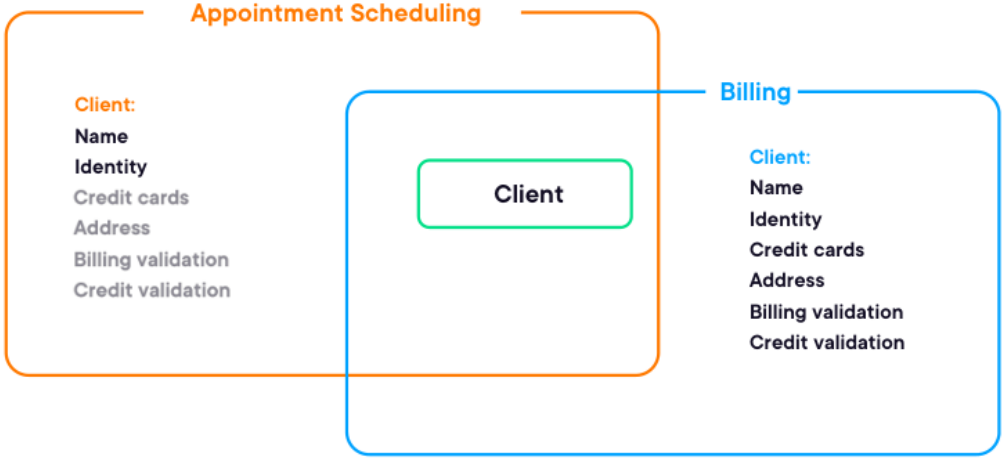


At least keep the concepts in mind for guidance

Not hard rules, but guidance

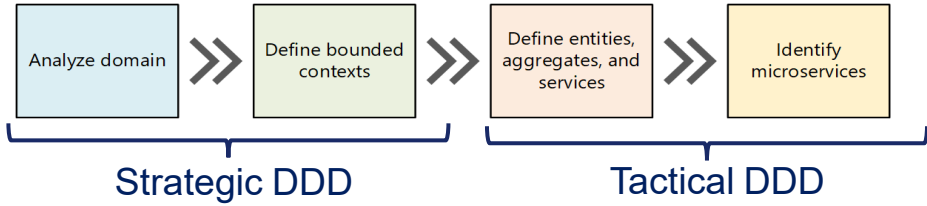
Let DDD guide you

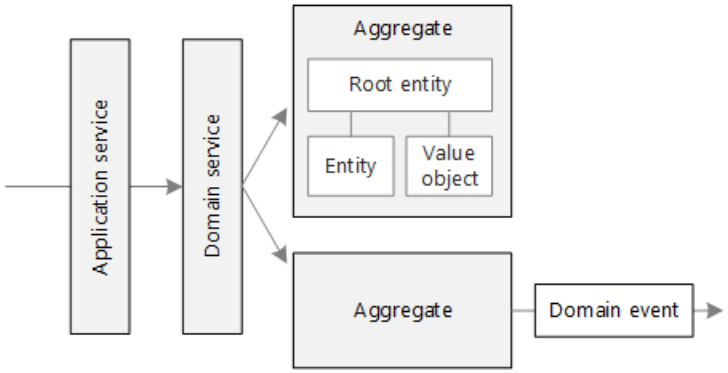
But What About Those Common Types?



Universidad de La Sabana
FACULTAD DE INGENIERIA

P3: Tactical DDD

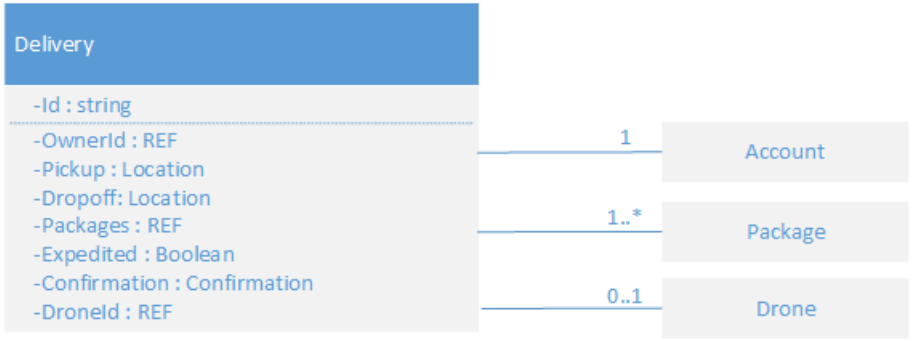


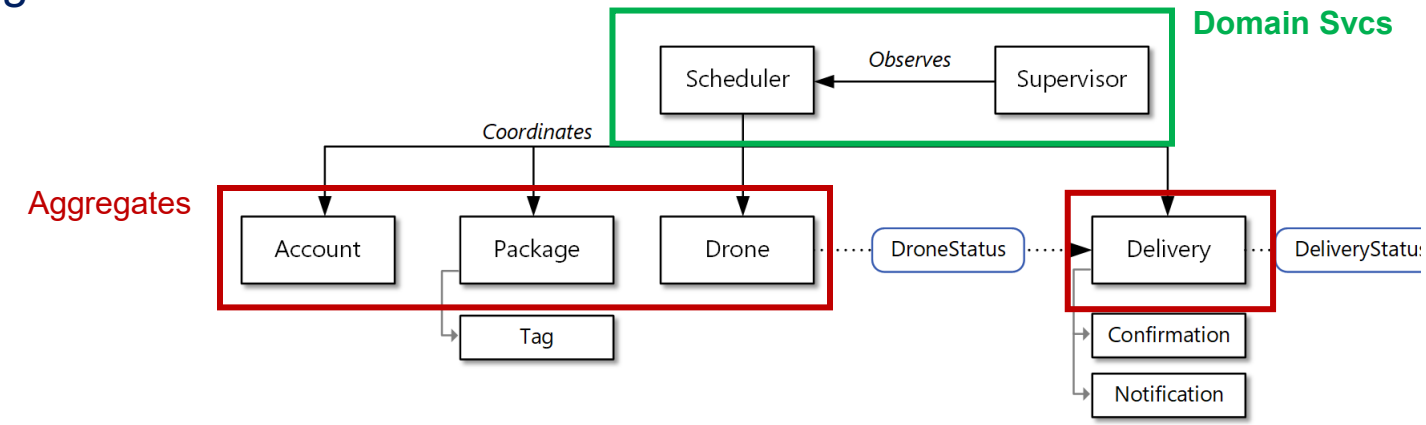


Define entities, aggregates and services

Value: Immutable Attributes, self-contained actions.
Entity: Mutable Attributes->lifecycle, GUID or autoinc, possible antipattern – anemic DD
Aggregate: Independent lifecycle, independent actions, transactional invariant (biz rules), eventual consistency through aggregates using Domain event.
Root Agg: Allowed entrance, encapsulate entities to do it consistent.
Domain events: Call by aggregate and call behavior (biz process) through services

Aggregate relations





Scheduler Agent Supervisor pattern on Azure, however I see like AWS Step Function

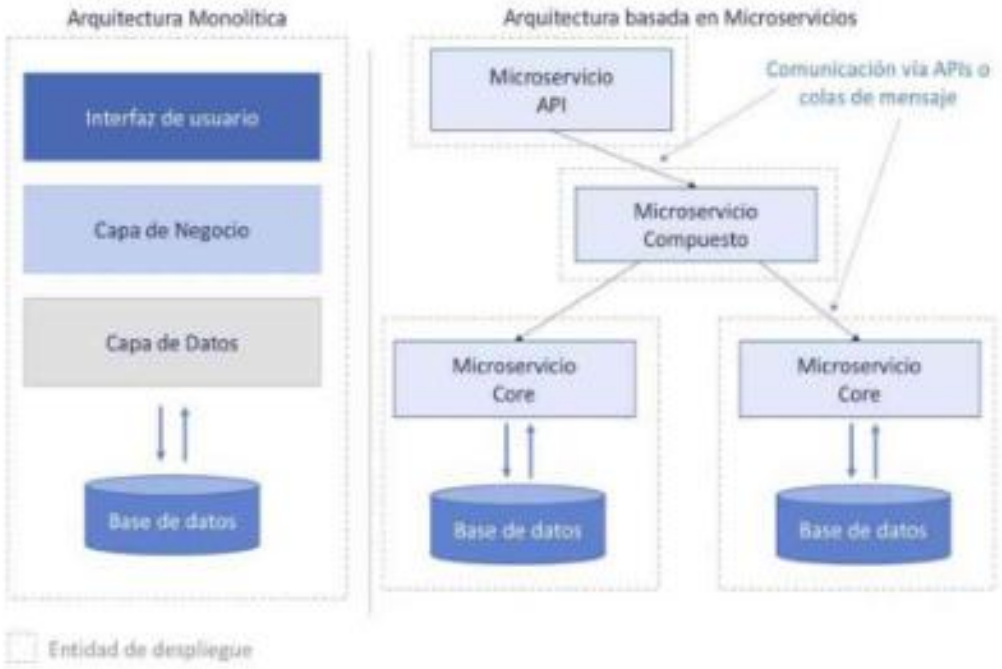


Figura 2.4. Arquitectura monolítica vs Microservicios

DS call AS ?
AS call DS ?

Domain Services: Domain layer, no store, orchestrate multiple aggregates.

Application Services: Service layer (Use Case, workflow), service façade, used the root aggregate to call biz functions, persistence by aggregates and call IS.

Infrastructure Services: Used by AS to have access to external tools.

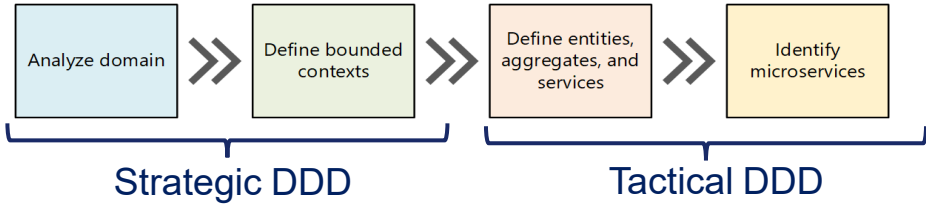
Composite / Aggregator Services: Aggregate data from others (multiple BC). No store.

Edge Services: Expose to external users with specific features (AuthN, AuthZ, rate limiting on API GW, Response limit on devices for BFF).

Sidecars: Transversal tools (monitoring, security)

Universidad de La Sabana
FACULTAD DE INGENIERIA

P3: Tactical DDD



DDD **Value Types**

- Collection of individual values
- Fully defined by its collection of attributes
- Immutable
- More precise and accurate than primitive types

DDD **Aggregates**

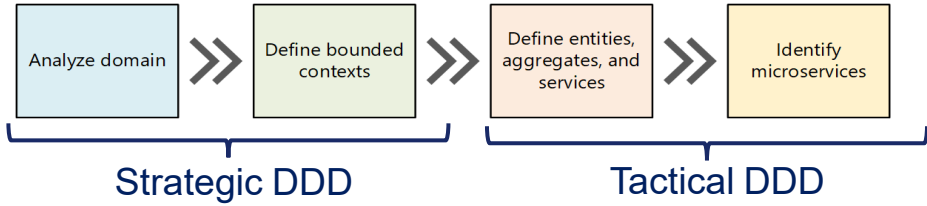
- A few individual entities constantly used and referenced together
- Cluster of associated objects treated as one for data changes
- Aggregate roots
- Preserve transactional integrity

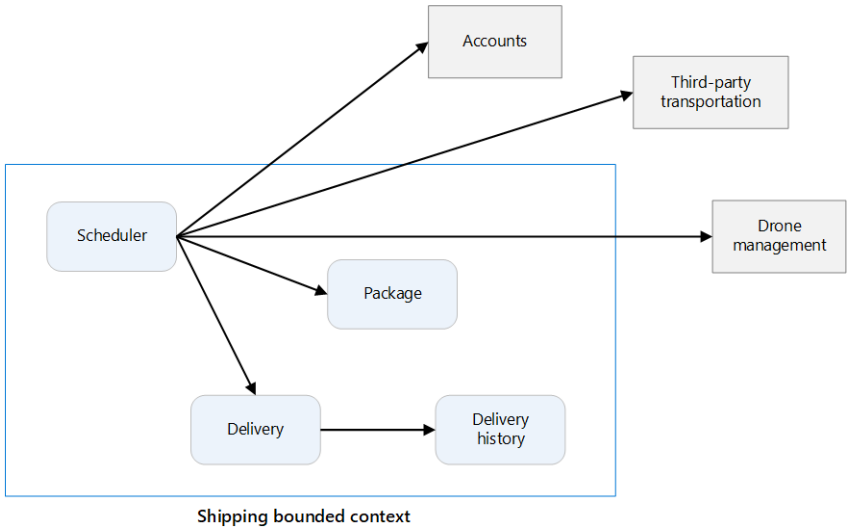
DDD **Entities**

- Need an identity
- Uniqueness is important to the specific object
- Typically made of data and behavior
- Contain domain logic, but not persistence logic

Universidad de La Sabana
FACULTAD DE INGENIERIA

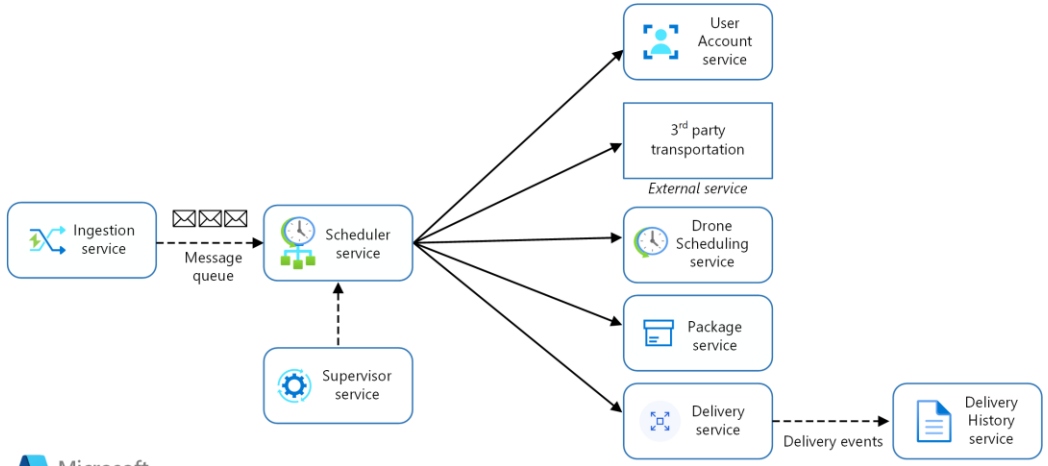
P4: Identity microservices





Microsoft Azure

As suggestion, create microservices per aggregate and per domain service.
Each microservice has a responsibility.



Microsoft Azure

[AzMs]

Universidad de La Sabana

FACULTAD DE INGENIERÍA

Study Cases



Universidad de La Sabana
FACULTAD DE INGENIERIA

Case 1: SMB – Logistic Company

- Development company (<50 IT employees).
- Role: Software Architect.
- Customer: Latam software company, which resell this software to seaports on many countries.
- Initial time: 4 months << Upselling and Cross-Selling.
- End time: 12 months.

- Strengths: **Ubiquitous Language** **(Pallet, Packet, Item, SKU, EDI/EDX Format, PO, Invoice, Shipment).**
- Challenges: Initial Scope and budget (Greenfield Project), Tight Deadlines, Team Building, UX/UI Design (Responsive), Domain Knowledge, Re-work, Mobile Farm for Testing.
- Features: BFF, Responsive (Tablets and Desktop), Multi-{country, currency, language}.

- Information: Questionaries, SME Talks (regular).

- Learned Lessons: Empirical microservices application and DDD method, explicit communication, keep friends since 2017, I lost the chance to learn Azure !.

Industries:
- Transport << JAlv
- Financial
- Healthcare
- Publishing
- AI-Customer

Universidad de La Sabana
FACULTAD DE INGENIERIA

Case 2: Bank - Credit Ecosystem

- National Bank (~4000 IT employees, Regional DNA).
- Role: Solutions Architect (Analista de Arquitectura I).
- Customer: Credit BU (People, SMB, Enterprise and Government).
- Initial time: 12 months.
- End time: 36 months (Estimated).

- Strengths: Team Building, City, Deadlines, Domain Knowledges, Processes.
- Challenges: Initial Scope (Brownfield Project), Interoperability, Time for querying on Specialized areas (SME != Domain Expert).
- Features: BPM, API managed, async processes.

- Information: **Event Storming (3 days)**, Domain Expert Reviews.

- Learned Lessons: Obtain the signature (promotion), first time with AWS and GCP on 2019, sell ideas to stubborn people is a wasted of time (PoV).

Universidad de La Sabana
FACULTAD DE INGENIERIA

Case 3: Bank – Enterprise Channel

- Digital Lab (~700 Digital Lab, Shared DNA)
- Role: Solutions Architect
- Customer: Capital Bank, part of Financial Holding
- Initial time: Unknown
- End time: 12 months (Estimated)

- Strengths: Processes, Roles, Team Building,
- Challenges: Tight Deadlines, Team Building (Outsource)
- Features: Edge Cloud, API managed, Synch and Async process (Sagas), Microfrontends (2 Experience).

- Information: Event Storming (1 days, virtual). Initial 12 Subdomains: Logon, Transfer, Customer Data, Investments, Payments, Transfer, Security, …

- Learned Lessons: Scope.

Universidad de La Sabana
FACULTAD DE INGENIERIA

References

- [BEAr] Ford, N., Parsons, et all (2022). Building evolutionary architectures: Support constant change, 2nd ed. O'Reilly Media, Inc.
- [EmMD] Khan, O. M. A., Siddiqui, N., & Oleson, T. (2021). *Embracing microservices design: A practical guide to revealing anti-patterns and architectural pitfalls to avoid microservices fallacies*. Packt Publishing.
- [MSPa] Pluralsight. (n.d.). Microservices architecture [Skill path. https://app.pluralsight.com/paths/skill/microservices-architecture-new
- [AzMs] Microsoft. (n.d.). Microservices architecture style. Azure Architecture Center. https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/microservices
- [DMHp] DDD Reference from Domain Language, Inc. (n.d.). Domain Language [Home page]. Retrieved July 28, 2026, from https://www.domainlanguage.com/ * Summary of Eric Evan's book.
- [MoSA] Esposito, D. (2020). Modern software architecture: Domain models, CQRS, and event sourcing [Video course]. Pluralsight. https://app.pluralsight.com/ilx/video-courses/modern-software-architecture-domain-models-cqrs-event-sourcing/course-overview
- [MsEi] Roldán Martínez, D., & Valderas Aranda, P. J. (2018). *Microservicios: Un enfoque integrado*. Ra-Ma Editorial.
- [MFAm] Fowler, M. (2003, November 25). *Anemic domain model*. MartinFowler.com. https://martinfowler.com/bliki/AnemicDomainModel.html





Universidad de La Sabana

FACULTAD DE INGENIERÍA

CONTACTO

Francisco Moreno

Profesor Catedra – Facultad de Ingeniería

franciscomodi@unisabana.edu.co